Linux initiation & Bash

Tous les cours en un seul PDF

10 chapitres Linux initiation et 6 chapitres Bash, dans l’ordre des deux parcours.

Cours complets issus du Markdown du coffre. Les activités HTML, les quiz et les TP restent accessibles sur le site.

Ouvrir le site et ses activités

Sommaire

Linux initiation

Chapitre 01 — Comprendre Linux **— p. 3**

Chapitre 02 — Installer la machine virtuelle **— p. 11**

Chapitre 03 — L'interface graphique **— p. 15**

Chapitre 04 — Le terminal **— p. 19**

Chapitre 05 — Fichiers et répertoires **— p. 25**

Chapitre 06 — L'arborescence Linux **— p. 30**

Chapitre 07 — Utilisateurs et permissions **— p. 35**

Chapitre 08 — Gestion des paquets **— p. 40**

Chapitre 09 — Redirections et tubes **— p. 44**

Chapitre 10 — Réseau et services **— p. 49**

Bash

Chapitre 01 — Le shell et son environnement **— p. 53**

Chapitre 02 — Les variables **— p. 59**

Chapitre 03 — Écrire son premier script **— p. 66**

Chapitre 04 — Les tests et les conditions **— p. 70**

Chapitre 05 — Les boucles **— p. 78**

Chapitre 06 — Traiter du texte avec sed et awk **— p. 84**

Chapitre 01 — Comprendre Linux

Vous utilisez peut-être déjà Linux sans le savoir : un téléphone Android, une box Internet ou un serveur qui héberge un site peuvent s'appuyer sur son noyau. Avant d'installer Debian, découvrons ce que recouvrent **Linux**, **GNU**, **logiciel libre** et **distribution**.

Objectifs À la fin du chapitre, vous pourrez expliquer le rôle d'un système d'exploitation, distinguer GNU de Linux, reconnaître les grandes familles de licences et comprendre pourquoi nous utilisons Debian.

Aucune commande à mémoriser pour le moment. L'objectif est de comprendre les mots que vous rencontrerez pendant la formation.

1. Qu'est-ce qu'un système d'exploitation ?

Un **système d'exploitation**, souvent abrégé **OS** (*Operating System*), fait fonctionner l'ordinateur et permet aux applications d'utiliser son matériel.

Quand vous ouvrez un document, le traitement de texte demande au système de lire le fichier sur le disque. Si vous lancez plusieurs applications, le système répartit entre elles le temps du processeur et la mémoire disponible. Il gère aussi les périphériques et les droits d'accès aux fichiers.

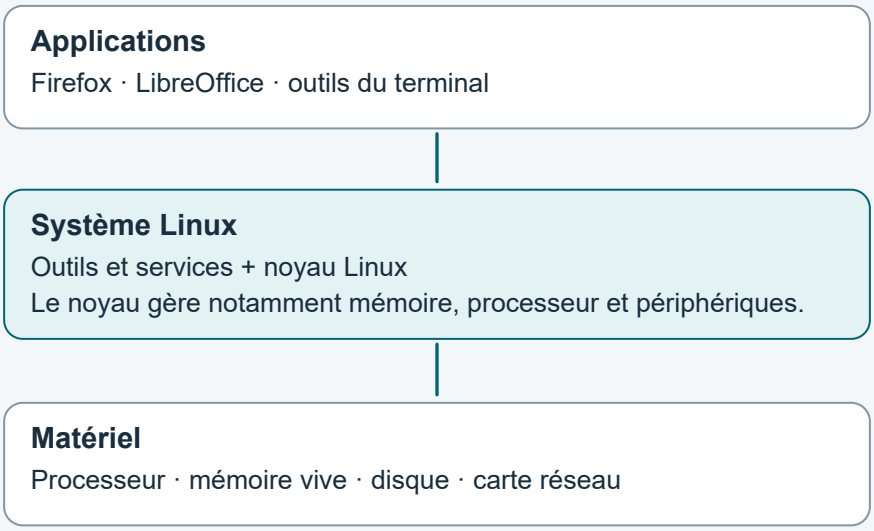
Une image pour comprendre Imaginez un **chef d'orchestre** : les applications demandent des ressources, le système organise leur utilisation et le matériel réalise les opérations.

illustration-01

01

Le rôle du système Linux

Des applications au matériel : chaque couche a un rôle.



Une distribution réunit le noyau, les outils et les logiciels nécessaires à son utilisation.

Le système Linux fait le lien entre les applications et le matériel.

Les applications s'appuient sur le système pour utiliser le matériel.

Vous connaissez probablement **Windows**, **macOS**, **Android** ou **iOS**. Sur un ordinateur, **Debian** et **Ubuntu** sont deux exemples de systèmes construits autour du noyau Linux.

Le **noyau** (*kernel*) est la partie centrale du système : il gère notamment le processeur, la mémoire et l'accès aux périphériques. Le navigateur, le bureau graphique et les commandes du terminal sont d'autres logiciels, qui s'appuient sur lui.

---

2. D'Unix à GNU/Linux : trois repères

Unix : les idées de départ

Unix apparaît en **1969** aux laboratoires Bell, notamment avec **Ken Thompson** et **Dennis Ritchie**. Il permet à plusieurs personnes d'utiliser un même ordinateur et d'y exécuter plusieurs programmes.
Histoire d'Unix — The Open Group

Unix a popularisé une organisation que vous retrouverez sous Linux : des utilisateurs avec des droits distincts, des fichiers rangés dans une arborescence et de petits outils que l'on peut combiner.

Linux est un système de type Unix : il reprend de nombreux principes d'Unix, mais son noyau a été développé indépendamment. Linux n'est donc pas simplement « une version gratuite d'Unix ».

GNU : Richard Stallman et le logiciel libre

En **1983**, **Richard Stallman** annonce le projet **GNU**, dont le développement commence en **1984**. Son objectif est de construire un système compatible avec Unix qui respecte la liberté de ses utilisateurs. Le nom signifie **GNU's Not Unix**, soit « GNU n'est pas Unix ». Annonce du projet GNU

Le projet fournit progressivement des outils essentiels : **Bash** pour interpréter les commandes, **GCC** pour transformer du code source en programmes et les **outils GNU** comme `ls` ou `cp`. Vous utiliserez certains de ces outils dans les prochains chapitres.

Richard Stallman fonde également la **Free Software Foundation (FSF)** en **1985**, pour soutenir le logiciel libre. GNU développe de nombreux composants, mais son propre noyau n'est pas encore prêt pour constituer le système recherché. Repères historiques — FSF

Linux : le noyau de Linus Torvalds

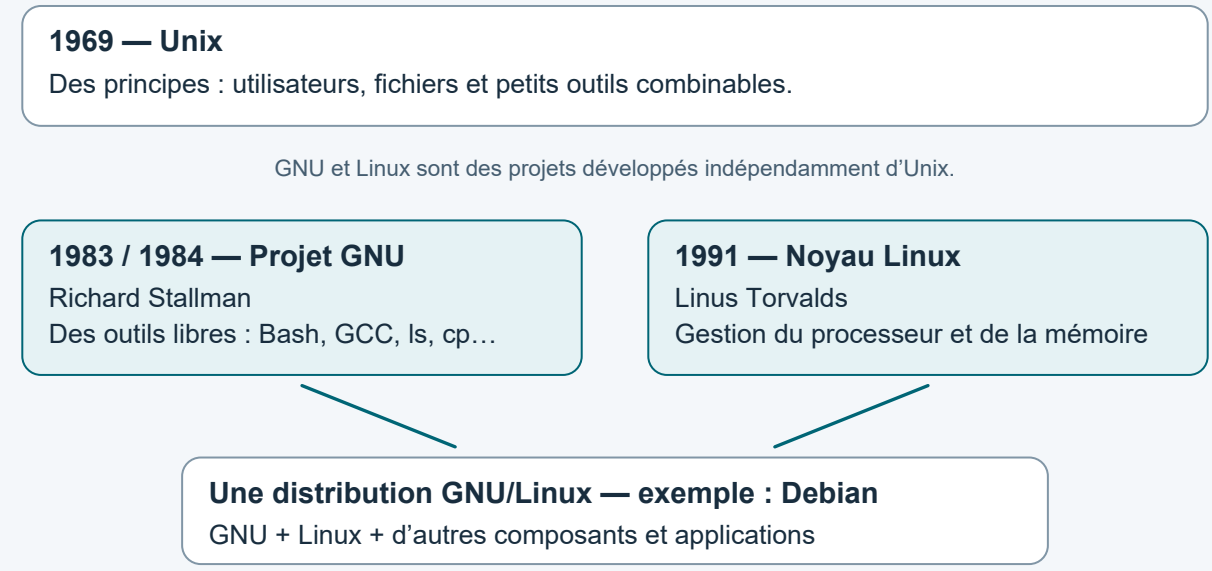
En **1991**, **Linus Torvalds**, alors étudiant finlandais, lance le noyau **Linux**. Des développeurs rejoignent ensuite le projet. Associé aux outils GNU et à d'autres logiciels, ce noyau permet de construire des systèmes complets.

L'expression **GNU/Linux** rappelle cette association. Dans la conversation courante, on dit souvent simplement « Linux » pour parler de l'ensemble. **Debian est une distribution GNU/Linux.** Tous les systèmes qui utilisent le noyau Linux n'utilisent cependant pas l'environnement GNU : Android en est un exemple. Présentation du système GNU

01

Unix, GNU, Linux : qui apporte quoi ?

Des principes hérités, des projets distincts et un système assemblé.



Ces flèches montrent un assemblage de composants, pas une filiation du code d'Unix.

Unix apporte des principes ; GNU apporte des outils libres ; Linux apporte le noyau. Une distribution assemble ces composants avec d'autres logiciels.

Deux personnes, deux contributions **Richard Stallman → GNU et les libertés du logiciel. Linus Torvalds → le noyau Linux.** Le système que nous allons utiliser est le résultat de ces projets et de nombreuses autres contributions.

---

3. Logiciel libre, open source et gratuit : quelle différence ?

Le **code source** est l'ensemble des instructions écrites par les développeurs pour construire un logiciel. Vous pouvez utiliser un logiciel sans savoir lire son code, comme vous pouvez utiliser une voiture sans être mécanicien.

Les quatre libertés du logiciel libre

Un **logiciel libre** donne à ses utilisateurs quatre libertés, traditionnellement numérotées de **0 à 3** :

Liberté	Ce que cela permet concrètement
0 — Utiliser	Exécuter le programme pour l'usage de son choix, personnel ou professionnel.
1 — Étudier et modifier	Comprendre son fonctionnement et l'adapter à ses besoins, grâce au code source.
2 — Partager des copies	Donner une copie du logiciel à un collègue ou à un proche.
3 — Partager ses modifications	Redistribuer une version améliorée pour que d'autres en profitent.

Ces libertés portent sur les **droits accordés**, pas sur le prix du logiciel. Définition du logiciel libre — GNU/FSF

L'open source : le code et les droits qui l'accompagnent

Un logiciel **open source** fournit l'accès à son code source avec une licence autorisant notamment son utilisation, sa modification et sa redistribution. Un code visible sur Internet, accompagné d'une interdiction de le modifier, ne suffit donc pas.

Les expressions **logiciel libre** et **open source** recouvrent largement les mêmes logiciels. La première met l'accent sur les libertés des utilisateurs ; la seconde sur l'ouverture du code et le développement collaboratif. L'**Open Source Initiative (OSI)** définit les critères d'une licence open source. Définition de l'open source — OSI

La recette et les permissions Le code source ressemble à une **recette de cuisine**. La licence précise si vous pouvez la cuisiner, la modifier et partager votre version. **Pouvoir lire la recette ne donne pas automatiquement tous ces droits.**

Gratuit ne veut pas dire libre

Un logiciel peut être **gratuit et propriétaire** : vous ne payez rien, mais vous n'avez pas nécessairement accès au code ni le droit de le modifier. À l'inverse, un logiciel libre peut être vendu, et des entreprises peuvent facturer son installation, son hébergement ou son assistance.

La transparence facilite l'étude du logiciel et permet à d'autres de poursuivre son développement. Elle ne garantit toutefois **ni l'absence de failles, ni des mises à jour éternelles**. La maintenance et la bonne configuration restent nécessaires.

---

4. Les licences libres : les règles du partage

Une **licence logicielle** est un texte qui précise les droits et les conditions d'utilisation, de modification et de redistribution d'un programme. Les auteurs conservent leurs droits d'auteur : publier un logiciel libre ne signifie pas abandonner tous leurs droits.

Vous n'avez pas besoin d'apprendre ces textes par cœur. Pour débuter, retenez surtout **deux grandes approches**.

Le copyleft : conserver les libertés lors du partage

La **GNU GPL** (*General Public License*) est une licence à **copyleft**. Lorsqu'une version modifiée d'un programme couvert par la GPL est distribuée, les destinataires doivent conserver les droits prévus par cette licence et pouvoir obtenir le code source correspondant selon ses conditions.

Exemple : une entreprise adapte un outil sous GPL et distribue cette version à ses clients. Elle doit respecter la GPL pour cette version et leur permettre d'obtenir son code source. Elle ne peut pas retirer les libertés que la licence leur accorde.

> Utiliser un programme GPL ne rend pas vos documents publics Rédiger un courrier ou utiliser un programme sous GPL ne vous oblige pas à publier votre travail. Une modification conservée pour un usage privé n'a pas non plus à être publiée simplement parce qu'elle existe. Les obligations de partage de la GPL concernent notamment la **distribution du logiciel couvert**, pas les documents que vous créez avec lui. FAQ des licences GNU

Le **noyau Linux est sous GPL version 2**. Les autres logiciels d'une distribution ont leurs propres licences : Debian n'est pas un unique programme placé sous une unique licence. Licence du noyau Linux

Les licences permissives : autoriser davantage de réutilisations

Les licences **MIT**, **BSD** et **Apache 2.0** autorisent une réutilisation large du code, y compris dans des logiciels propriétaires, sous réserve de respecter leurs conditions. Elles imposent notamment de conserver certains avis de licence et de droits d'auteur. Apache 2.0 comprend aussi des dispositions explicites sur les brevets.

Approche	Exemples	Idée principale
Copyleft	GNU GPL	Préserver les libertés sur le programme couvert et ses versions dérivées distribuées.
Permissive	MIT, BSD, Apache 2.0	Faciliter les réutilisations, sans imposer que toute version dérivée reste libre.

Exemple : une entreprise peut intégrer un composant sous MIT dans une application propriétaire en respectant la licence, notamment les mentions à conserver. Les utilisateurs de l'application complète ne reçoivent pas forcément le code de cette application. Licence MIT — OSI, licence Apache 2.0

Le réflexe utile Avant de redistribuer un logiciel ou de réutiliser son code, cherchez son fichier `LICENSE`, `COPYING` ou sa rubrique **À propos** → **Licence**. Un téléchargement gratuit ne remplace pas la lecture des permissions.

5. Une distribution : le système prêt à utiliser

Une **distribution Linux** assemble un noyau Linux, des outils système, des bibliothèques nécessaires aux programmes, un gestionnaire de paquets et des applications. Elle peut aussi proposer un bureau graphique.

Pour **Debian**, on peut se représenter cet assemblage ainsi :

```
Noyau Linux + outils GNU + autres composants et applications
                        ↓
              Distribution Debian
```

L'équipe d'une distribution choisit les composants, prépare leur installation et fournit des mises à jour. Vous n'avez pas à assembler chaque pièce vous-même.

Famille ou projet	Exemples	Repère pour la formation
Debian	Debian, Ubuntu, Linux Mint	Ubuntu s'appuie sur Debian ; l'édition principale de Mint s'appuie sur Ubuntu.
Red Hat	Fedora, RHEL, Rocky Linux, AlmaLinux	Un autre écosystème, fréquent dans les environnements professionnels.
SUSE	openSUSE, SUSE Linux Enterprise	Un écosystème disposant de ses propres outils d'administration.
Autres projets	Arch Linux, Kali Linux	Des choix ou usages particuliers ; Kali, basée sur Debian, vise notamment les tests de sécurité.

Notre choix : Debian Nous utiliserons **Debian** pour apprendre sur une base stable, documentée et utilisable aussi bien avec un bureau que comme serveur. Les principes étudiés vous serviront ensuite

dans d’autres distributions.

Une distribution peut contenir des composants sous différentes licences, et éventuellement des éléments propriétaires, comme certains micrologiciels nécessaires au matériel. **Utiliser Linux ne signifie donc pas que chaque composant installé est libre.**

---

6. Bureau ou serveur : deux usages du même système

Un **poste de travail** sert directement à une personne : naviguer sur le Web, écrire des documents ou consulter ses fichiers. Un **serveur** fournit des services à d’autres machines, par exemple des pages Web ou un espace de partage.

Interface	Comment l’utiliser ?	Exemple
Bureau graphique	Fenêtres, icônes, menus et souris.	Ouvrir le gestionnaire de fichiers et créer un dossier.
Terminal	Saisir des commandes au clavier.	Demander l’affichage du contenu d’un dossier.

Ces interfaces accèdent au **même système et aux mêmes fichiers**. Vous pouvez ouvrir un terminal dans un bureau graphique ; il n’est pas nécessaire de choisir définitivement entre les deux.

Un serveur peut fonctionner sans bureau pour économiser des ressources et limiter les logiciels à maintenir. Dans cette formation, nous commencerons avec une interface graphique, puis nous découvrirons progressivement le terminal.

---

7. Vidéo et échange

Regardez la vidéo proposée, puis utilisez le cours pour répondre aux questions ci-dessous. Sa lecture nécessite une connexion Internet.

Vidéo du cours : ouvrir la vidéo en ligne

Ouvrir la vidéo sur YouTube

1. Qui a lancé GNU ? Qui a créé le noyau Linux ?
2. Un logiciel gratuit est-il forcément libre ? Expliquez avec les droits de l’utilisateur.
3. Quelle différence principale retenez-vous entre une licence GPL et une licence MIT ?
4. Pourquoi Debian est-elle une distribution plutôt qu’un simple noyau ?

Afficher les pistes de réponse

1. Richard Stallman a lancé GNU ; Linus Torvalds a créé le noyau Linux.

2. Non. Le prix ne dit pas si l’on peut étudier, modifier et redistribuer le programme.

3. La GPL préserve les libertés sur les versions dérivées distribuées du programme couvert. MIT permet aussi des réutilisations propriétaires, à condition de respecter ses mentions et conditions.

4. Debian assemble le noyau avec des outils, des applications et un système de gestion et de mise à jour des logiciels.

À retenir

Mot	L'idée essentielle
Unix	Un système historique dont Linux reprend de nombreux principes.
GNU	Le projet libre lancé par Richard Stallman, à l'origine de nombreux outils système.
Linux	Le noyau lancé par Linus Torvalds en 1991.
GNU/Linux	L'association du noyau Linux et de l'environnement GNU, avec d'autres composants.
Libre / open source	Des droits sur l'utilisation, le code, les modifications et le partage ; pas une promesse de gratuité.
Licence	Le texte qui fixe ces droits et leurs conditions : GPL, MIT, BSD, Apache…
Distribution	Un ensemble prêt à installer, comme Debian ou Ubuntu.

Vous pouvez maintenant faire le quiz de 20 questions, puis passer au chapitre 02 — Installer la VM.

Chapitre 02 — Installer la machine virtuelle

1. Qu'est-ce qu'une machine virtuelle ?

Une **machine virtuelle** (VM) est un ordinateur simulé à l'intérieur de ton ordinateur réel. Elle fonctionne comme un vrai PC mais dans une fenêtre — elle a son propre disque dur, sa propre mémoire, son propre système d'exploitation.

> **Analogie** : une machine virtuelle, c'est comme une maison dans une maison. Tu construis un appartement à l'intérieur de ta propre maison. L'appartement est indépendant — ce qui s'y passe n'affecte pas le reste de la maison.

Avantages de la VM pour la formation : - On peut tout casser sans risque — redémarrer suffit à repartir de zéro - Pas besoin d'un second ordinateur pour pratiquer Linux - Isolée du système principal (Windows, macOS)

illustration-02

02

Comprendre la machine virtuelle

Le système hôte et le système invité sont deux environnements distincts.

PC physique — système hôte

Windows, Linux ou macOS : il continue de fonctionner.

VirtualBox — logiciel de virtualisation

Machine virtuelle — système invité Debian

Son propre système, ses comptes et ses fichiers
RAM et processeur partagés avec le PC physique
Disque virtuel : un fichier VDI sur le disque de l’hôte

Pour revenir à un état antérieur : restaurer un instantané créé auparavant. Redémarrer ne suffit pas.

Le PC hôte exécute VirtualBox, qui fait fonctionner le système invité Debian.

Le PC hôte exécute VirtualBox, qui fait fonctionner le système invité Debian.

2. VirtualBox — l'outil de virtualisation

VirtualBox est le logiciel qui permet de créer et de gérer des machines virtuelles. Il est gratuit et fonctionne sous Windows, macOS et Linux.

Téléchargement : https://www.virtualbox.org/wiki/Downloads

---

3. Télécharger Debian

Nous utilisons **Debian 12** (ou 13 si disponible), en version **netinstall** (installation minimale par réseau) ou **DVD ISO**.

Téléchargement officiel : https://www.debian.org/download

---

4. Créer la machine virtuelle dans VirtualBox

Étape 1 — Nouveau

Dans VirtualBox : **Nouvelle** (bouton en haut à gauche)

Paramètres à saisir :

Champ	Valeur
Nom	`Debian-Formation` (ou au choix)
Type	Linux
Version	Debian (64-bit)

Étape 2 — Mémoire vive

Alloue **2 Go minimum**, 4 Go si ta machine le permet.

> Ne pas allouer plus de la moitié de la RAM totale de ton ordinateur — sinon ton PC hôte sera ralenti.

Étape 3 — Disque dur virtuel

- Type : **VDI** (VirtualBox Disk Image)
- Stockage : **dynamique** (le fichier ne grandit que si nécessaire)
- Taille : **20 Go minimum**

Étape 4 — Réseau

Dans **Configuration > Réseau** : - **Carte 1** : NAT (accès Internet via l'hôte)

5. Installer Debian

Démarrer l'installation

1. Sélectionne ta VM → **Démarrer**
2. VirtualBox te demande de choisir l'ISO → sélectionne le fichier Debian téléchargé
3. Le programme d'installation démarre

Référence visuelle : https://www.server-world.info/en/note?os=Debian_13&p=install

Déroulement de l'installation

Étape	Ce qu'il faut faire
Langue	Français
Pays	France
Clavier	Français
Nom de la machine	`debian` (ou au choix, sans espaces)
Domaine	Laisser vide
Mot de passe root	Choisir et noter
Nouvel utilisateur	Créer un compte standard (ex : `stagiaire`)
Partitionnement	**Assisté — utiliser un disque entier** (option simple)
Logiciels	Cocher **Environnement de bureau Debian** + **Utilitaires standard**
GRUB	Installer sur le disque principal (`/dev/sda`)

L'installation dure environ 10 à 20 minutes selon la connexion. À la fin, la VM redémarre automatiquement sur Debian.

6. Premier démarrage

Au démarrage, un écran de connexion apparaît. Connecte-toi avec le compte utilisateur créé pendant l'installation (pas root).

En règle générale, on **ne travaille pas en tant que** `root` au quotidien. Le compte root est réservé aux opérations d'administration. Utilise ton compte utilisateur normal pour commencer.

Récapitulatif des commandes utiles (VirtualBox)

Action	Comment faire
Démarrer la VM	Sélectionner la VM → Démarrer
Mettre en pause	Menu Machine → Mettre en pause
Éteindre proprement	Dans la VM : Menu → Arrêter le système
Prendre un instantané	Menu Machine → Prendre un instantané (sauvegarde de l'état)
Redimensionner l'écran	Affichage → Écran virtuel → Ajuster à la fenêtre

À retenir

- Une **machine virtuelle** est un ordinateur simulé dans l'ordinateur — on peut tout casser sans risque.
- **VirtualBox** est le logiciel de virtualisation utilisé dans cette formation (gratuit, multiplateforme).
- Créer une VM : nom, type Linux/Debian, **2 Go de RAM minimum**, disque VDI dynamique **20 Go**.
- L'installation Debian dure 10 à 20 min : langue, utilisateur, partitionnement assisté, interface graphique.
- Ne jamais se connecter en **root** pour le travail courant — utiliser le compte utilisateur normal.
- **Prendre un instantané** avant toute manipulation risquée permet de revenir à l'état précédent.

TP associé → `tp-02-installer-la-vm.md`

Chapitre 03 — L'interface graphique

1. Les environnements de bureau

Sous Linux, l'interface graphique n'est pas intégrée au système — c'est un composant séparé appelé **environnement de bureau**. Il en existe plusieurs, chacun avec son style et ses outils.

Environnement	Style	Points forts
GNOME	Moderne, épuré	Simple, très répandu (Ubuntu, Debian)
KDE Plasma	Riche, personnalisable	Ressemble à Windows, nombreuses options
XFCE	Léger, classique	Idéal pour les vieilles machines
MATE	Classique	Stabilité, peu gourmand

Dans cette formation, nous utilisons **GNOME**, l'environnement par défaut de Debian Desktop.

2. Découvrir GNOME

Le bureau

Au premier démarrage sous GNOME, le bureau est intentionnellement minimaliste — pas d'icônes par défaut sur le bureau.

Les éléments principaux :

Élément	Emplacement	Rôle
Barre supérieure	Haut de l'écran	Heure, réseau, son, paramètres système
Activités	Bouton en haut à gauche	Ouvrir le menu des applications et voir les fenêtres ouvertes
Dock	Bas ou côté gauche	Applications favorites en accès rapide

Ouvrir une application

Deux méthodes : 1. Cliquer sur **Activités** → taper le nom de l'application 2. Cliquer sur **Activités** → grille d'applications (9 points)

---

3. Les applications courantes sous Linux

Linux dispose d'équivalents libres pour la plupart des logiciels courants :

Besoin	Windows	Équivalent Linux
Navigateur web	Chrome, Edge	Firefox, Chromium
Bureautique	Microsoft Office	LibreOffice (Writer, Calc, Impress)
Email	Outlook	Thunderbird
Images	Paint, Photoshop	GIMP, Eye of GNOME
Lecteur vidéo	Windows Media Player	VLC
Gestionnaire de fichiers	Explorateur Windows	Nautilus (Fichiers)
Terminal	Invite de commandes	GNOME Terminal, Konsole

---

4. Le gestionnaire de fichiers (Nautilus)

Nautilus (appelé simplement "Fichiers") est l'équivalent de l'Explorateur Windows.

Il permet de : - Naviguer dans les dossiers - Créer, copier, déplacer, renommer des fichiers - Accéder aux disques et périphériques

> Le dossier personnel de l'utilisateur est `/home/ton_nom/`. C'est l'équivalent du dossier `C:\Users\Nom` sous Windows. C'est là que tu trouves tes Documents, Images, Téléchargements…

---

5. Paramètres système

Menu Activités → **Paramètres** (icône engrenage) :

- **Réseau** : configurer Wi-Fi, Ethernet
- **Affichage** : résolution, disposition des écrans
- **Son** : volume, périphériques audio
- **Utilisateurs** : gérer les comptes

• **Mises à jour** : vérifier les mises à jour système

6. Ouvrir un terminal

Le terminal est l'outil central de l'administration Linux. Depuis GNOME, plusieurs façons de l'ouvrir :

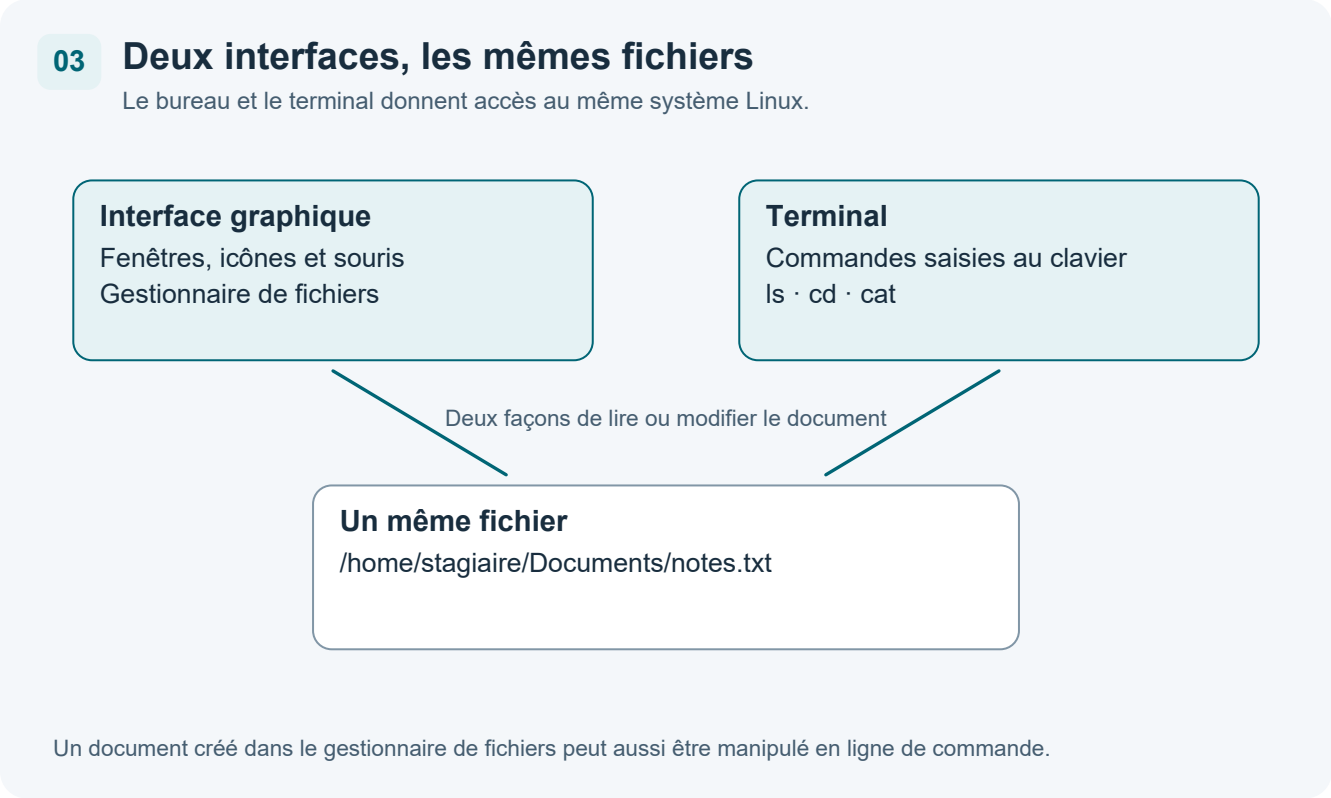
• **Activités** → taper `terminal` → Entrée

• Clic droit sur le bureau → **Ouvrir dans un terminal** (si disponible)

• Raccourci : `Ctrl + Alt + T` (selon la configuration)

À partir du module suivant, on travaille exclusivement dans le terminal. L'interface graphique est utile au quotidien, mais le terminal est indispensable pour l'administration.

illustration-03



Le bureau et le terminal manipulent les mêmes fichiers.

Le bureau et le terminal manipulent les mêmes fichiers.

Récapitulatif

Notion	À retenir
Environnement de bureau	Interface graphique de Linux (GNOME, KDE, XFCE…)
GNOME	Environnement par défaut de Debian, moderne et épuré
Nautilus	Gestionnaire de fichiers GNOME
`/home/utilisateur/`	Dossier personnel de l'utilisateur
Terminal	Outil en ligne de commande — indispensable pour l'administration

À retenir

- L'interface graphique sous Linux s'appelle un **environnement de bureau** — elle est séparée du système.
- **GNOME** est l'environnement par défaut de Debian : bureau épuré, menu Activités, Dock.
- Linux dispose d'équivalents libres pour tous les logiciels courants (LibreOffice, Firefox, VLC…).
- Le dossier personnel de l'utilisateur est `/home/ton_nom/`, accessible via le raccourci `~`.
- Le **terminal** s'ouvre depuis Activités → taper "terminal". C'est l'outil central de l'administration.

Chapitre 04 — Le terminal

1. Pourquoi utiliser le terminal ?

L'interface graphique est pratique pour un usage quotidien. Mais le terminal permet de faire des choses **impossibles ou très lentes** avec une souris :

- Automatiser des tâches répétitives
- Administrer un serveur distant (sans interface graphique)
- Exécuter des opérations sur des centaines de fichiers en une seule commande
- Diagnostiquer des problèmes système

Analogie : l'interface graphique, c'est comme cuisiner avec des plats préparés — pratique et rapide. Le terminal, c'est cuisiner soi-même — plus de temps au départ, mais tu peux faire exactement ce que tu veux.

2. Le prompt

Quand tu ouvres le terminal, tu vois une ligne qui ressemble à ça :

```
stagiaire@debian:~$
```

Ce **prompt** t'indique que le terminal est prêt. Il affiche :

Partie	Signification
`stagiaire`	Ton nom d'utilisateur
`@`	"at" (chez)
`debian`	Le nom de la machine
`:`	Séparateur
`~`	Le répertoire courant (~ = ton dossier personnel)
`$`	Tu es un utilisateur normal
`#`	Tu es root (administrateur) — attention !

3. Structure d'une commande

Toute commande Linux suit la même structure :

```
commande [options] [arguments]
```

Partie	Rôle	Exemple
`commande`	Ce qu'on veut faire	`ls`
`options`	Comment le faire (commence par `-`)	`-l`, `-a`, `--help`
`arguments`	Sur quoi agir	`/home`, `fichier.txt`

Exemple complet :

```
ls -l /home
```

→ lister (`ls`) en détail (`-l`) le contenu de `/home`

> Linux est **sensible à la casse** — `ls` et `LS` sont deux choses différentes. Toujours écrire les commandes en minuscules.

---

4. Se repérer dans le système

`pwd` — Où suis-je ?

```
pwd
# /home/stagiaire
```

`pwd` (*Print Working Directory*) affiche le chemin complet du dossier où tu te trouves.

`ls` — Qu'y a-t-il ici ?

```
ls             # liste simple
ls -l          # liste détaillée (permissions, taille, date)
ls -a          # affiche aussi les fichiers cachés (commençant par .)
ls -lh         # liste détaillée avec tailles lisibles (Ko, Mo…)
ls /etc        # liste le contenu d'un autre dossier
```

`cd` — Se déplacer

```
cd /etc            # aller dans /etc
cd ..              # remonter d'un dossier
cd                 # revenir dans son dossier personnel (~)
cd ~/Documents     # aller dans Documents (~ = dossier personnel)
cd -               # retourner dans le dossier précédent
```

> **Astuce** : appuyer sur **Tab** complète automatiquement le nom d'un fichier ou dossier. Si plusieurs choix sont possibles, appuyer deux fois sur Tab affiche les possibilités.

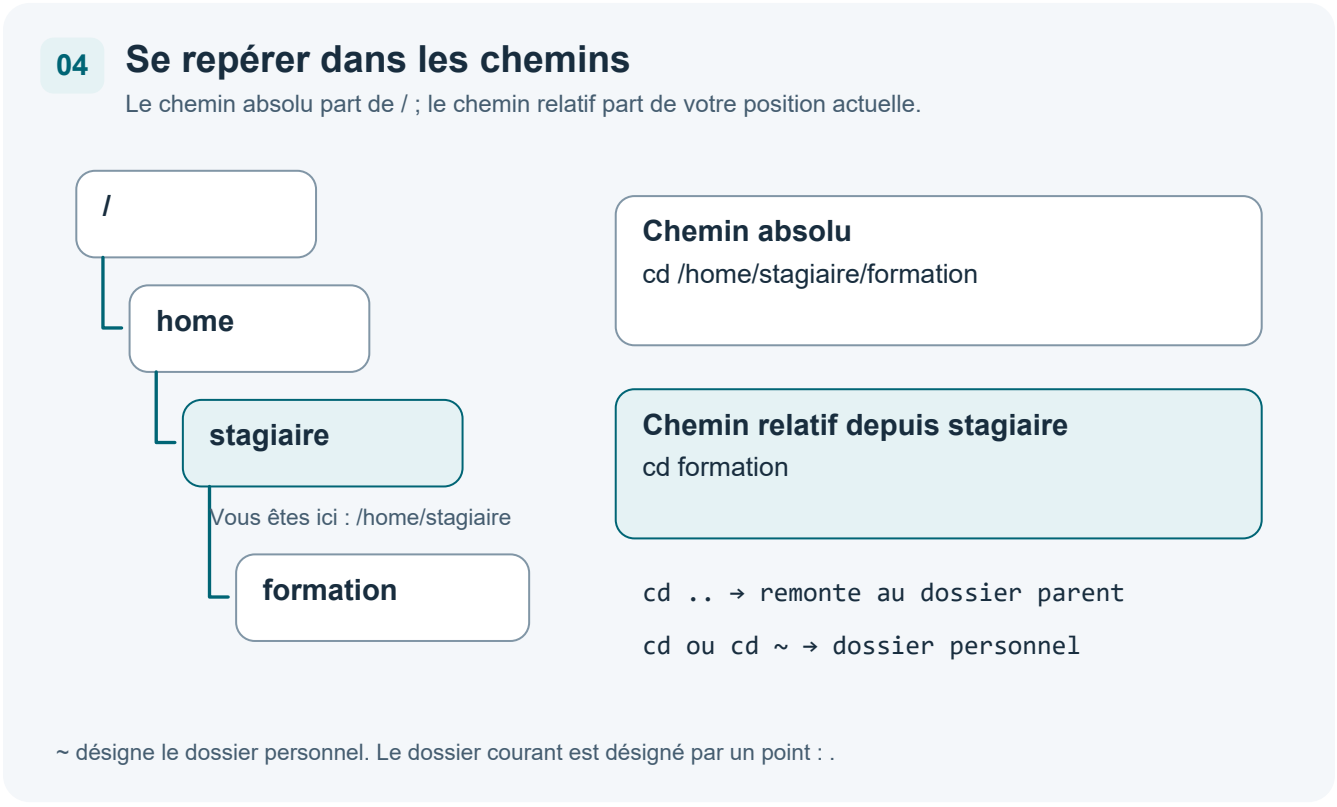
5. Chemins absolus et relatifs

Il existe deux façons d'indiquer un emplacement :

Type	Description	Exemple
Absolu	Part toujours de la racine /	`/home/stagiaire/Documents`
Relatif	Part du dossier courant	`Documents` ou `../autres`

```
# Si je suis dans /home/stagiaire :
cd Documents          # chemin relatif → va dans /home/stagiaire/Documents
cd /home/stagiaire/Documents   # chemin absolu → même résultat
```

illustration-04



Deux chemins différents peuvent mener au même dossier.

Deux chemins différents peuvent mener au même dossier.

6. Lire des fichiers

```
cat fichier.txt        # affiche tout le contenu d'un coup
less fichier.txt       # affiche page par page (q pour quitter, flèches pour naviguer)
head fichier.txt       # affiche les 10 premières lignes
tail fichier.txt       # affiche les 10 dernières lignes
tail -f /var/log/syslog  # affiche les nouvelles lignes en temps réel
```

---

7. Obtenir de l'aide

```
man ls          # manuel complet de la commande ls (q pour quitter)
ls --help       # aide résumée directement dans le terminal
```

> `man` (*manual*) est le manuel intégré de Linux. Chaque commande y est documentée. C'est la référence officielle.

---

8. Raccourcis indispensables du terminal

Raccourci	Action
`Tab`	Complétion automatique du nom
`↑` / `↓`	Naviguer dans l'historique des commandes
`Ctrl + C`	Interrompre une commande en cours
`Ctrl + L`	Effacer l'écran (équivalent de `clear`)
`Ctrl + A`	Aller au début de la ligne
`Ctrl + E`	Aller à la fin de la ligne
`history`	Afficher l'historique des commandes

Récapitulatif des commandes

Commande	Description
`pwd`	Affiche le répertoire courant
`ls`	Liste le contenu d'un répertoire
`ls -l`	Liste détaillée (permissions, taille, date)
`ls -a`	Affiche aussi les fichiers cachés
`ls -lh`	Liste détaillée avec tailles lisibles
`cd /chemin`	Va dans le dossier spécifié
`cd ..`	Remonte d'un niveau
`cd`	Retourne dans le dossier personnel
`cd -`	Retourne dans le dossier précédent
`cat fichier`	Affiche le contenu d'un fichier
`less fichier`	Affiche un fichier page par page
`head fichier`	Affiche les 10 premières lignes
`tail fichier`	Affiche les 10 dernières lignes
`man commande`	Affiche le manuel de la commande
`commande --help`	Affiche l'aide résumée
`clear`	Efface l'écran
`history`	Affiche l'historique des commandes

À retenir

- La structure d'une commande est : `commande [options] [arguments]`.
- Linux est **sensible à la casse** — toujours écrire les commandes en minuscules.
- On navigue avec `pwd` (où suis-je ?), `ls` (qu'y a-t-il ici ?), `cd` (je me déplace).
- Un chemin **absolu** part de `/` ; un chemin **relatif** part du dossier courant.
- On lit les fichiers avec `cat`, `less`, `head`, `tail`.
- L'aide est toujours disponible : `man commande` ou `commande --help`.
- **Tab** complète automatiquement — à utiliser sans modération.

TP associé → `tp-04-terminal.md`

Chapitre 05 — Fichiers et répertoires

1. Tout est fichier sous Linux

Sous Linux, **tout est représenté comme un fichier** : les documents, les dossiers, les périphériques (clé USB, disque dur), les connexions réseau… C'est l'un des principes fondateurs du système.

> **Analogie** : imagine que dans ta maison, chaque chose — la lampe, le robinet, la télé — soit rangée dans un tiroir étiqueté. Linux fait pareil : tout a un chemin et un nom dans l'arborescence.

2. Créer des fichiers et des dossiers

Créer un dossier — `mkdir`

```
mkdir projets                  # crée le dossier "projets" ici
mkdir /home/stagiaire/projets  # chemin absolu
mkdir -p projets/linux/cours   # crée toute la hiérarchie en une fois
```

> L'option `-p` (*parents*) crée tous les dossiers intermédiaires s'ils n'existent pas.

Créer un fichier vide — `touch`

```
touch notes.txt          # crée un fichier vide (ou met à jour la date si existant)
touch a.txt b.txt c.txt  # crée plusieurs fichiers d'un coup
```

Écrire dans un fichier — `echo`

```
echo "Bonjour Linux !" > notes.txt   # écrit (écrase si existant)
echo "Ligne 2" >> notes.txt          # ajoute à la fin
```

3. Copier, déplacer, renommer

Copier — `cp`

```
cp notes.txt notes-sauvegarde.txt    # copie dans le même dossier
cp notes.txt /tmp/                   # copie dans un autre dossier
cp -r projets/ /tmp/                 # copie un dossier et tout son contenu (-r = récursif)
```

> Sans l'option `-r`, `cp` ne peut pas copier un dossier — il faut toujours `-r` pour les dossiers.

Déplacer ou renommer — `mv`

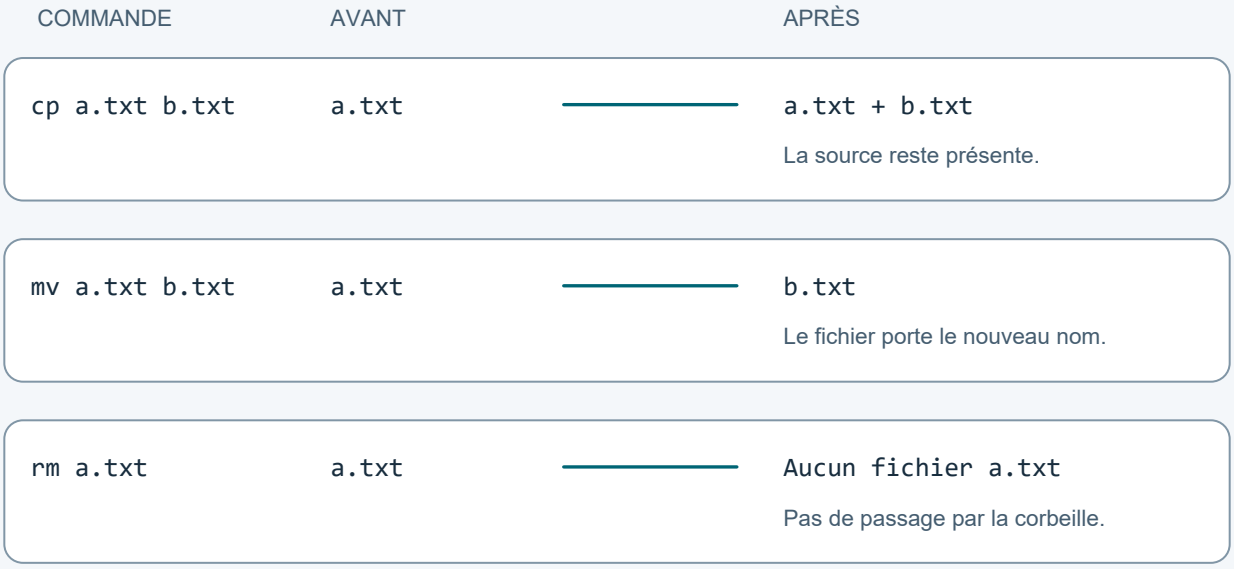
```
mv notes.txt documents/           # déplace dans le dossier documents
mv notes.txt rapport.txt          # renomme le fichier
mv projets/ /home/stagiaire/work/ # déplace un dossier entier
```

`mv` fait à la fois **déplacer** et **renommer** — c'est la même commande. Si la destination est dans le même dossier avec un nouveau nom, c'est un renommage.

illustration-05

05 **Copier, déplacer, supprimer**

Observer ce qui reste après chaque commande.



Exemples indépendants : b.txt n'existe pas au départ. Vérifier la cible avant toute suppression.

Comparer le résultat de cp, mv et rm.

Comparer le résultat de cp, mv et rm.

---

4. Supprimer

Supprimer un fichier — `rm`

```
rm notes.txt          # supprime le fichier
rm a.txt b.txt c.txt  # supprime plusieurs fichiers
```

Supprimer un dossier — `rm -r` ou `rmdir`

```
rmdir projets/         # supprime un dossier VIDE uniquement
rm -r projets/         # supprime un dossier et tout son contenu
```

`rm` **est définitif.** Il n'y a pas de corbeille en ligne de commande — ce qui est supprimé est perdu. Vérifier toujours avant d'appuyer sur Entrée.

`rm -rf /` ou `rm -rf *` peuvent détruire tout le système. Ne jamais les taper sans être absolument certain de ce qu'on fait.

---

5. Rechercher des fichiers — `find`

```
find /home -name "notes.txt"          # cherche par nom exact
find /home -name "*.txt"              # cherche tous les .txt
find /etc -name "*.conf" -type f      # cherche les fichiers .conf uniquement
find /tmp -mtime -1                   # fichiers modifiés dans les dernières 24h
```

---

6. Les jokers (wildcards)

Les jokers permettent de cibler plusieurs fichiers à la fois :

Joker	Signification	Exemple
`*`	N'importe quelle suite de caractères	`*.txt` → tous les fichiers .txt
`?`	Un seul caractère	`fichier?.txt` → fichier1.txt, fichierA.txt…
`[abc]`	Un caractère parmi la liste	`fichier[123].txt`

```
ls *.txt          # tous les fichiers .txt
rm rapport_*.txt  # supprime tous les fichiers commençant par "rapport_"
cp *.conf /tmp/   # copie tous les .conf dans /tmp
```

---

7. Afficher et éditer le contenu d'un fichier

Voir le contenu — rappel

```
cat fichier.txt     # tout afficher
less fichier.txt    # page par page
```

Éditeur de texte en terminal — `nano`

`nano` est l'éditeur le plus simple pour débuter :

```
nano notes.txt    # ouvre le fichier (le crée s'il n'existe pas)
```

Raccourcis nano :

Raccourci	Action
`Ctrl + O`	Sauvegarder
`Ctrl + X`	Quitter
`Ctrl + K`	Couper la ligne
`Ctrl + U`	Coller
`Ctrl + W`	Rechercher

Récapitulatif des commandes

Commande	Description
`mkdir dossier`	Crée un dossier
`mkdir -p a/b/c`	Crée une hiérarchie de dossiers
`touch fichier`	Crée un fichier vide
`echo "texte" > fichier`	Écrit dans un fichier (écrase)
`echo "texte" >> fichier`	Ajoute à la fin d'un fichier
`cp source destination`	Copie un fichier
`cp -r dossier destination`	Copie un dossier
`mv source destination`	Déplace ou renomme
`rm fichier`	Supprime un fichier
`rm -r dossier`	Supprime un dossier et son contenu
`rmdir dossier`	Supprime un dossier vide
`find /chemin -name "motif"`	Recherche des fichiers
`nano fichier`	Ouvre un fichier dans l'éditeur nano

À retenir

- `mkdir` crée un dossier, `touch` crée un fichier vide, `nano` édite un fichier.
- `cp` copie (ajouter `-r` pour un dossier), `mv` déplace ou renomme, `rm` supprime.
- `rm` **est irréversible** — pas de corbeille en ligne de commande. Vérifier avant d'appuyer sur Entrée.

- `echo "texte" >` écrase un fichier ; `echo "texte" >>` ajoute à la fin.
- `find /chemin -name "motif"` recherche des fichiers dans l'arborescence.
- Les **jokers** permettent de cibler plusieurs fichiers : `*` (tout), `?` (1 caractère).
- Dans `nano` : `Ctrl+O` pour sauvegarder, `Ctrl+X` pour quitter.

TP associé → `tp-05-fichiers.md`

Chapitre 06 — L'arborescence Linux

1. Tout part de la racine

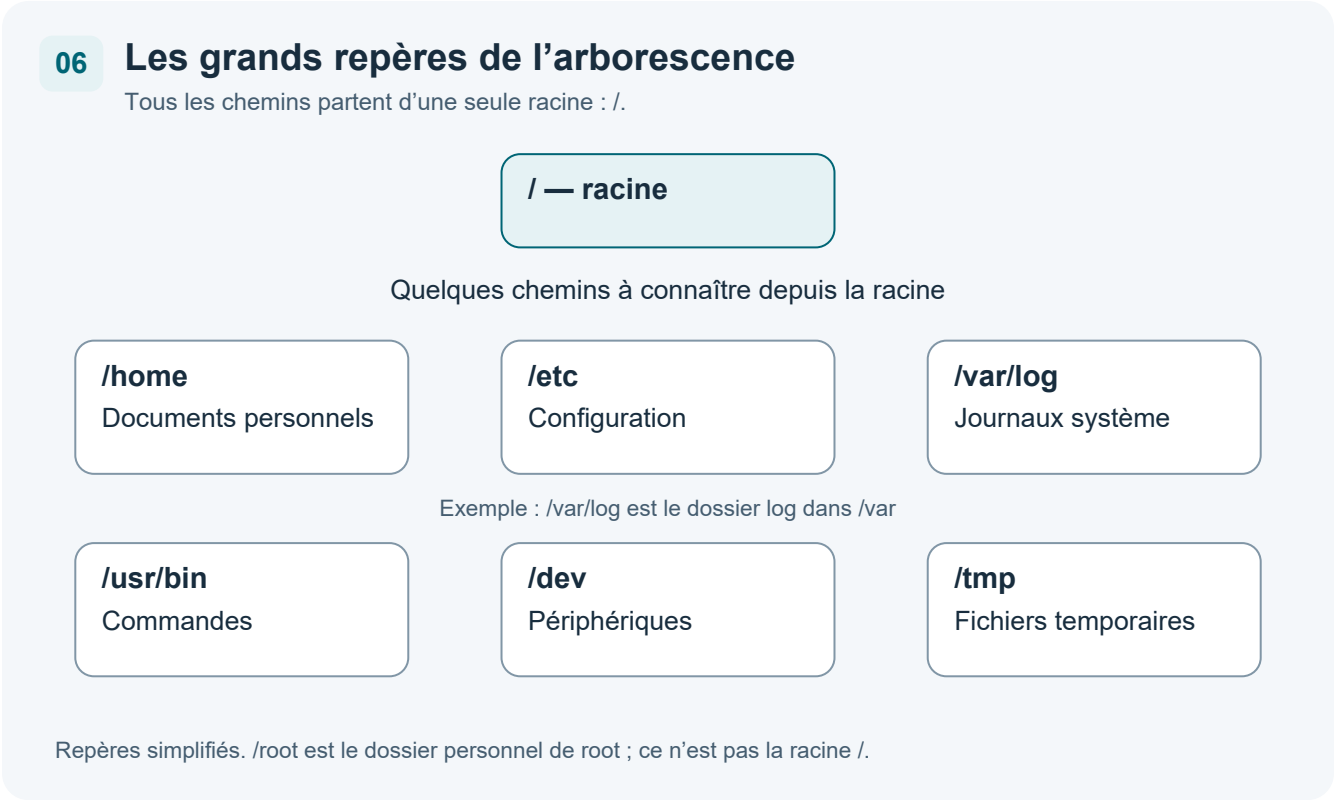
Sous Linux, il n'y a **pas de lettre de lecteur** (pas de `C:\`, `D:\` comme Windows). Il y a une seule **racine**, notée `/`, et tout le système de fichiers part de là.

```
/
├── etc/       ← configuration
├── home/      ← dossiers personnels des utilisateurs
├── var/       ← données variables (logs, bases de données…)
├── tmp/       ← fichiers temporaires
├── usr/       ← programmes installés
├── bin/       ← commandes essentielles
├── boot/      ← fichiers de démarrage
├── dev/       ← périphériques
└── ...
```

Analogie : l'arborescence Linux, c'est comme un immeuble avec des étages bien organisés. La racine `/` est le rez-de-chaussée, et chaque dossier est un étage avec un rôle précis.

Cette organisation suit le standard **FHS** (*Filesystem Hierarchy Standard*), adopté par toutes les distributions Linux.

illustration-06



Quelques repères pour retrouver les dossiers essentiels.

---

2. Les dossiers essentiels à connaître

`/home` — les dossiers personnels

Chaque utilisateur a son propre dossier dans `/home` :

```
/home/
├── stagiaire/    ← dossier de l'utilisateur "stagiaire"
├── alice/
└── bob/
```

`~` est un raccourci pour ton dossier personnel. `~/Documents` = `/home/stagiaire/Documents`.

`/etc` — la configuration du système

Contient les fichiers de configuration de tous les logiciels installés.

```
ls /etc
# passwd   hosts   fstab   apt/   ssh/   ...
```

Exemples importants :

Fichier	Rôle
`/etc/passwd`	Liste des utilisateurs du système
`/etc/hosts`	Correspondance noms ↔ adresses IP locales
`/etc/fstab`	Points de montage des disques
`/etc/apt/`	Configuration du gestionnaire de paquets

`/var` — les données variables

Contient les données qui changent en permanence : logs, bases de données, files d'attente…

```
/var/
├── log/       ← journaux système
│   ├── syslog
│   └── auth.log
├── cache/     ← cache des applications
└── lib/       ← données persistantes des applications
```

```
tail -f /var/log/syslog    # voir les logs en temps réel
```

`/tmp` — les fichiers temporaires

Espace de travail temporaire. **Tout son contenu est effacé au redémarrage.**

```
ls /tmp    # souvent quelques fichiers de session
```

> Ne jamais stocker des données importantes dans `/tmp` — elles seront perdues au prochain redémarrage.

`/usr` — les programmes

Contient la majorité des programmes installés et leurs ressources.

```
/usr/
├── bin/       ← commandes utilisateur (ls, cp, grep…)
├── sbin/      ← commandes administrateur
├── lib/       ← bibliothèques partagées
└── share/     ← données partagées (docs, icônes…)
```

`/bin` et `/sbin` — les commandes essentielles

Contiennent les commandes indispensables au démarrage du système (`ls`, `cp`, `bash`…). Sur les systèmes modernes, `/bin` est souvent un lien symbolique vers `/usr/bin`.

`/dev` — les périphériques

Représentation des périphériques matériels sous forme de fichiers :

```
ls /dev/
# sda   sda1   sda2   tty   null   zero   ...
```

Fichier	Représente
`/dev/sda`	Premier disque dur
`/dev/sda1`	Première partition du disque
`/dev/null`	"Trou noir" — jette tout ce qu'on y envoie
`/dev/tty`	Terminal courant

`/boot` — le démarrage

Contient le noyau Linux et les fichiers nécessaires au démarrage (GRUB).

```
ls /boot
# vmlinuz-6.1.0   initrd.img-6.1.0   grub/
```

---

3. Comparer avec Windows

Linux	Windows	Contenu
`/home/utilisateur/`	`C:\Users\Nom\`	Dossier personnel
`/etc/`	`C:\Windows\System32\` (en partie)	Configuration système
`/var/log/`	Observateur d'événements	Journaux système
`/tmp/`	`C:\Windows\Temp\`	Fichiers temporaires
`/usr/bin/`	`C:\Program Files\`	Programmes installés
`/dev/sda`	`C:`, `D:`	Disques et partitions

---

4. Commandes utiles pour explorer

```
df -h             # espace disque disponible sur chaque partition
du -sh /var/log   # taille du dossier /var/log
lsblk             # liste les disques et partitions
findmnt           # affiche les points de montage actifs
```

Récapitulatif des dossiers clés

Dossier	Contient
`/`	Racine — tout part de là
`/home/`	Dossiers personnels des utilisateurs
`/etc/`	Fichiers de configuration
`/var/log/`	Journaux système
`/tmp/`	Fichiers temporaires (effacés au redémarrage)
`/usr/bin/`	Programmes installés
`/dev/`	Périphériques matériels
`/boot/`	Noyau et fichiers de démarrage
`~`	Raccourci pour le dossier personnel

À retenir

- Sous Linux, tout part de la **racine** `/` — pas de lettre de lecteur comme Windows.
- Cette organisation suit le standard **FHS** adopté par toutes les distributions.
- `/home/` : dossiers personnels des utilisateurs — `~` est le raccourci du dossier courant.
- `/etc/` : **configuration** de tous les logiciels installés.
- `/var/log/` : **journaux** système, à consulter pour diagnostiquer des problèmes.
- `/tmp/` : fichiers **temporaires**, effacés à chaque redémarrage — ne jamais y stocker des données.
- `/usr/bin/` : la majorité des **programmes** installés.
- `df -h` affiche l'espace disque, `du -sh` la taille d'un dossier.

TP associé → `tp-06-arborescence.md`

Chapitre 07 — Utilisateurs et permissions

1. Pourquoi des utilisateurs et des permissions ?

Linux est un système **multi-utilisateurs** : plusieurs personnes peuvent utiliser la même machine, chacune avec son propre espace et ses propres droits. Les permissions garantissent que chaque utilisateur ne peut accéder qu'à ce qui lui appartient ou lui est autorisé.

> **Analogie** : imagine un immeuble. Chaque locataire a sa clé (son compte). Il peut entrer dans son appartement (son `/home`) mais pas chez les voisins. Le gardien (root) a un passe-partout.

2. Les utilisateurs

Le compte root

`root` est le **superutilisateur** — il a tous les droits sur le système. Il peut tout lire, modifier, supprimer.

> On n'utilise **jamais root pour le travail courant**. Une erreur en root peut détruire le système entier. On utilise un compte normal et on élève les privilèges uniquement quand nécessaire.

Les comptes utilisateurs

Chaque personne a son propre compte avec un nom, un mot de passe et un dossier personnel.

```
whoami        # affiche le nom de l'utilisateur courant
id            # affiche l'UID, le GID et les groupes
who           # liste les utilisateurs connectés
```

La commande sudo

`sudo` (*Superuser Do*) permet d'exécuter une commande **avec les droits root** sans se connecter en tant que root :

```
sudo apt update          # mise à jour avec les droits admin
sudo nano /etc/hosts     # modifier un fichier système
```

Linux demande ton mot de passe pour confirmer.

3. Les groupes

Un **groupe** rassemble plusieurs utilisateurs qui partagent les mêmes droits sur certaines ressources.

```
groups              # affiche les groupes de l'utilisateur courant
```

```
cat /etc/group      # liste tous les groupes du système
```

---

4. Gérer les utilisateurs

Ces commandes nécessitent les droits root (`sudo`) :

```
# Créer un utilisateur
sudo adduser alice          # interactif — demande le mot de passe et les infos

# Changer le mot de passe
sudo passwd alice

# Ajouter un utilisateur à un groupe
sudo usermod -aG sudo alice    # donne les droits sudo à alice

# Supprimer un utilisateur
sudo deluser alice
sudo deluser --remove-home alice    # supprime aussi son dossier personnel
```

---

5. Les permissions

Lire les permissions

```
ls -l
```

Exemple de sortie :

```
-rw-r--r--  1  stagiaire  stagiaire  1234  28 avr  notes.txt
drwxr-xr-x  2  stagiaire  stagiaire  4096  28 avr  Documents/
```

Décomposons la première colonne `-rw-r--r--` :

```
- rw- r-- r--
| |   |   └── autres utilisateurs : lecture seule
| |   └────── groupe : lecture seule
| └────────── propriétaire : lecture + écriture
└──────────── type : - fichier  d dossier  l lien
```

Les trois droits

Lettre	Droit	Sur un fichier	Sur un dossier
`r`	**read** — lire	Lire le contenu	Lister le contenu
`w`	**write** — écrire	Modifier le contenu	Créer / supprimer des fichiers
`x`	**execute** — exécuter	Lancer le fichier	Entrer dans le dossier

Les trois catégories

Catégorie	Symbole	Signification
Propriétaire	u (*user*)	La personne qui possède le fichier
Groupe	g (*group*)	Les membres du groupe associé
Autres	o (*others*)	Tous les autres utilisateurs

---

6. Modifier les permissions — `chmod`

Mode symbolique (facile à lire)

```
chmod u+x script.sh      # ajoute le droit d'exécution au propriétaire
chmod g-w fichier.txt    # retire le droit d'écriture au groupe
chmod o+r rapport.txt    # donne le droit de lecture aux autres
chmod a+x script.sh      # donne l'exécution à tous (a = all)
```

Mode numérique (rapide et précis)

Chaque droit a une valeur : `r=4`, `w=2`, `x=1`

On additionne les valeurs pour chaque catégorie :

Valeur	Droits
7	`rwx` (4+2+1)
6	`rw-` (4+2)
5	`r-x` (4+1)
4	`r--` (4)
0	`---` (rien)

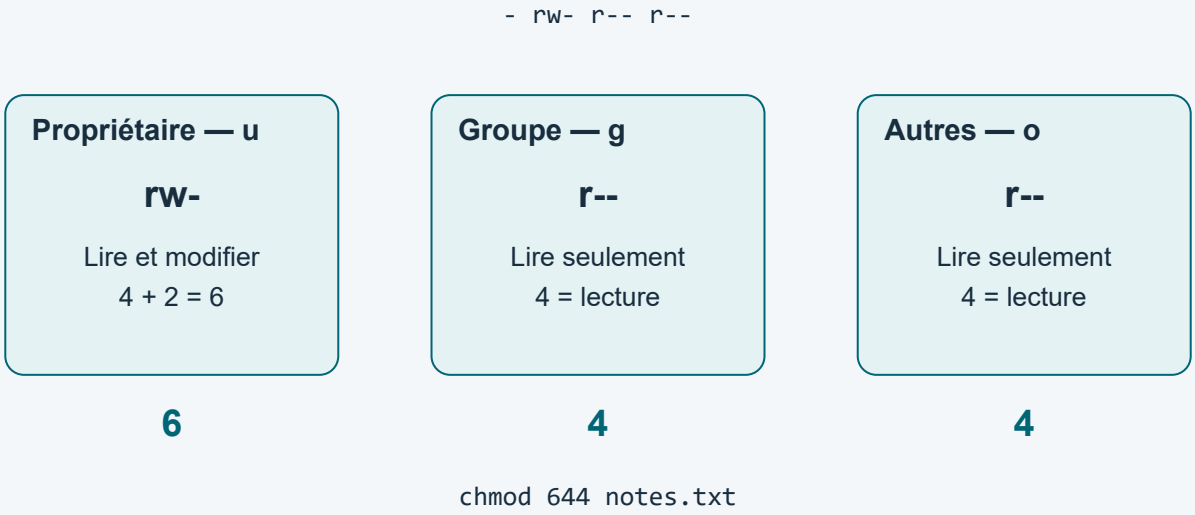
```
chmod 644 fichier.txt    # rw-r--r-- (standard pour un fichier)
chmod 755 script.sh      # rwxr-xr-x (standard pour un script)
chmod 700 secret.txt     # rwx------ (uniquement pour le propriétaire)
```

illustration-07

07

Lire les permissions d’un fichier

Trois catégories distinctes : propriétaire, groupe et autres utilisateurs.



r = 4, w = 2, x = 1. Pour un fichier lisible et modifiable uniquement par son propriétaire : chmod 600.

Lire les trois blocs de permissions et leur valeur numérique.

Lire les trois blocs de permissions et leur valeur numérique.

---

7. Changer le propriétaire — `chown`

```
sudo chown alice fichier.txt          # change le propriétaire
sudo chown alice:stagiaires fichier.txt  # change propriétaire ET groupe
sudo chown -R alice /home/alice/      # récursif sur tout un dossier
```

---

8. Exemples concrets

```
# Créer un script et le rendre exécutable
touch monscript.sh
chmod +x monscript.sh

# Protéger un fichier — lecture seule pour tout le monde
chmod 444 confidentiel.txt

# Donner accès à un dossier à tout le monde
chmod 755 partage/

# Vérifier qui possède un fichier
ls -l fichier.txt
```

Récapitulatif des commandes

Commande	Description
`whoami`	Affiche l'utilisateur courant
`id`	Affiche UID, GID et groupes
`groups`	Liste les groupes de l'utilisateur
`who`	Liste les utilisateurs connectés
`sudo commande`	Exécute avec les droits root
`sudo adduser nom`	Crée un nouvel utilisateur
`sudo passwd nom`	Change le mot de passe
`sudo usermod -aG groupe nom`	Ajoute un utilisateur à un groupe
`sudo deluser nom`	Supprime un utilisateur
`ls -l`	Affiche les permissions des fichiers
`chmod permissions fichier`	Modifie les permissions
`sudo chown user fichier`	Change le propriétaire

À retenir

- Chaque fichier appartient à un **utilisateur** et à un **groupe**, et possède des droits d'accès.
- **Ne jamais travailler en root** au quotidien — utiliser `sudo` pour les opérations ponctuelles.
- Les droits se lisent en trois blocs : propriétaire (`u`), groupe (`g`), autres (`o`).
- Les trois droits : `r` lire, `w` écrire, `x` exécuter (fichier) / entrer (dossier).
- `chmod` modifie les permissions — mode symbolique (`u+x`) ou numérique (`755`, `644`).
- `chown` change le propriétaire d'un fichier ou dossier.
- Valeurs numériques : `r=4`, `w=2`, `x=1` — ex : `7 = rwx`, `6 = rw-`, `4 = r--`.

TP associé → `tp-07-permissions.md`

Chapitre 08 — Gestion des paquets

1. Qu'est-ce qu'un paquet ?

Sous Linux, les logiciels sont distribués sous forme de **paquets** — des archives contenant le programme, ses dépendances et ses fichiers de configuration.

> **Analogie** : un paquet Linux, c'est comme un meuble IKEA dans sa boîte. Tout ce qu'il faut est dedans (pièces + notice + chevilles). Le gestionnaire de paquets, c'est comme le livreur qui va chercher la boîte à l'entrepôt (dépôt) et la livre chez toi (ton système).

---

2. Les dépôts (repositories)

Un **dépôt** est un serveur officiel qui héberge les paquets. `apt` sait où chercher grâce aux fichiers de configuration dans `/etc/apt/`.

```
cat /etc/apt/sources.list      # liste des dépôts configurés
ls /etc/apt/sources.list.d/    # dépôts supplémentaires
```

---

3. APT — le gestionnaire de paquets Debian/Ubuntu

`apt` (*Advanced Package Tool*) est l'outil de gestion des logiciels sur Debian et Ubuntu. Il télécharge, installe, met à jour et supprime des paquets automatiquement.

Mettre à jour la liste des paquets disponibles

```
sudo apt update
```

> `apt update` ne met **pas** à jour les logiciels — il met à jour la **liste** des versions disponibles. C'est toujours la première étape avant d'installer ou de mettre à jour.

Mettre à jour les logiciels installés

```
sudo apt upgrade           # met à jour tous les paquets installés
sudo apt full-upgrade      # idem + gère les changements de dépendances
```

Installer un logiciel

```
sudo apt install vim           # installe vim
sudo apt install vim curl git  # installe plusieurs paquets d'un coup
```

Supprimer un logiciel

```
sudo apt remove vim          # supprime le programme (garde la config)
sudo apt purge vim           # supprime le programme ET sa configuration
sudo apt autoremove          # supprime les paquets devenus inutiles
```

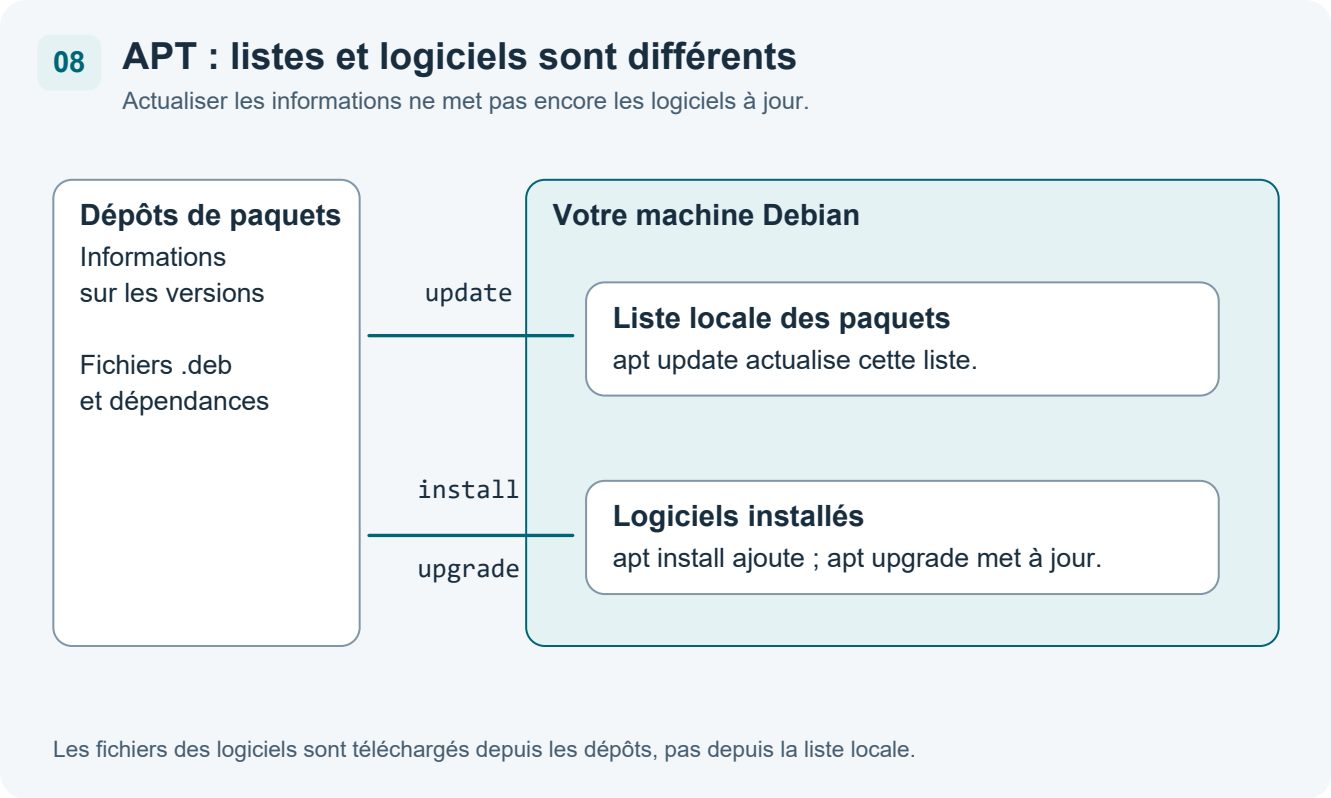
Chercher un logiciel

```
apt search editeur-texte    # cherche un paquet par mot-clé
apt show vim                # affiche les détails d'un paquet
```

Lister les paquets installés

```
apt list --installed          # tous les paquets installés
apt list --installed | grep vim  # filtrer par nom
```

illustration-08



apt update actualise les listes ; install et upgrade agissent sur les logiciels.

apt update actualise les listes ; install et upgrade agissent sur les logiciels.

---

4. La séquence type

Pour installer un logiciel proprement, toujours faire dans cet ordre :

```
sudo apt update           # 1. Mettre à jour la liste
sudo apt install nom      # 2. Installer
```

Pour faire la maintenance du système régulièrement :

```
sudo apt update && sudo apt upgrade    # met à jour tout
sudo apt autoremove                    # nettoie les paquets inutiles
```

---

5. dpkg — travailler avec des fichiers .deb

Parfois on télécharge directement un fichier `.deb` (équivalent d'un `.exe` Windows) depuis un site web. On utilise alors `dpkg` :

```
sudo dpkg -i fichier.deb      # installe le paquet
sudo dpkg -l | grep nom       # vérifie si un paquet est installé
```

---

6. Exemples d'installations utiles

```
sudo apt install curl         # outil de téléchargement en ligne de commande
sudo apt install git          # gestionnaire de versions
sudo apt install htop         # moniteur système interactif
sudo apt install tree         # affiche l'arborescence en forme d'arbre
sudo apt install net-tools    # outils réseau (ifconfig, netstat…)
sudo apt install vim          # éditeur de texte avancé
```

Récapitulatif des commandes

Commande	Description
`sudo apt update`	Met à jour la liste des paquets disponibles
`sudo apt upgrade`	Met à jour les paquets installés
`sudo apt install nom`	Installe un paquet
`sudo apt remove nom`	Supprime un paquet (garde la config)
`sudo apt purge nom`	Supprime un paquet et sa configuration
`sudo apt autoremove`	Supprime les paquets inutiles
`apt search mot-clé`	Recherche un paquet
`apt show nom`	Affiche les détails d'un paquet
`apt list --installed`	Liste les paquets installés
`sudo dpkg -i fichier.deb`	Installe un fichier .deb téléchargé

À retenir

- Un **paquet** est une archive contenant un logiciel + ses dépendances + sa configuration.
- Les paquets sont hébergés sur des **dépôts** (serveurs officiels) configurés dans `/etc/apt/`.
- `apt update` met à jour la **liste** des paquets disponibles — ce n'est pas une mise à jour des logiciels.
- `apt upgrade` met à jour les **logiciels** installés.
- `apt install nom` installe, `apt remove nom` désinstalle (garde la config), `apt purge` supprime tout.
- `apt autoremove` nettoie les paquets devenus inutiles.
- Toujours faire `apt update` avant `apt install`.

TP associé → `tp-08-paquets.md`

Chapitre 09 — Redirections et tubes

1. Les trois flux de données

Chaque commande Linux travaille avec trois canaux de communication :



Flux	Numéro	Défaut	Description
`stdin`	0	Clavier	Ce que reçoit la commande
`stdout`	1	Écran	Le résultat normal
`stderr`	2	Écran	Les messages d'erreur

Analogie : imagine une machine à café. Tu mets des grains (stdin), tu obtiens du café (stdout), et si quelque chose cloche la machine affiche une erreur (stderr). Les redirections te permettent de changer d'où viennent les grains et où va le café.

2. Rediriger la sortie vers un fichier

`>` — écrire dans un fichier (écrase)

```
ls /etc > liste.txt         # sauvegarde le résultat dans liste.txt
date > date_du_jour.txt     # écrit la date dans un fichier
```

`>` **écrase** le fichier s'il existe déjà. Son contenu précédent est perdu.

`>>` — ajouter à la fin d'un fichier

```
echo "Ligne 1" > journal.txt    # crée le fichier avec "Ligne 1"
echo "Ligne 2" >> journal.txt   # ajoute "Ligne 2" à la fin
echo "Ligne 3" >> journal.txt   # ajoute "Ligne 3" à la fin
cat journal.txt
# Ligne 1
# Ligne 2
# Ligne 3
```

`2>` — rediriger les erreurs

```
ls /dossier-inexistant 2> erreurs.txt   # redirige l'erreur dans un fichier
ls /etc /dossier-inexistant &> tout.txt # redirige résultat ET erreurs
```

Jeter les erreurs — `/dev/null`

```
ls /dossier-inexistant 2> /dev/null    # l'erreur disparaît silencieusement
```

`/dev/null` est le "trou noir" de Linux — tout ce qu'on y envoie est supprimé.

---

3. Les tubes (pipes) — `|`

Le tube `|` connecte la **sortie d'une commande** à l'**entrée de la suivante**. C'est l'outil le plus puissant du terminal Linux.

> **Analogie** : un tube de plomberie. L'eau sort d'un robinet (commande 1) et entre directement dans le seau (commande 2) sans toucher le sol.

```
commande1 | commande2 | commande3
```

Exemples

```
# Compter les fichiers dans /etc
ls /etc | wc -l

# Chercher un mot dans la liste des processus
ps aux | grep firefox

# Voir les 5 premières lignes de la liste des paquets installés
apt list --installed | head -5

# Chercher un utilisateur dans /etc/passwd
cat /etc/passwd | grep stagiaire

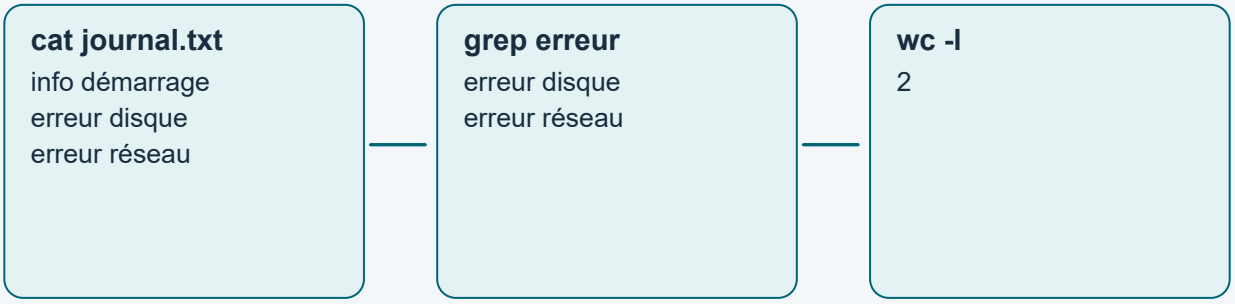
# Combiner tri et comptage
ls /etc | sort | uniq | wc -l
```

illustration-09

09

Suivre les données dans un tube

Chaque commande reçoit le résultat de la commande précédente.




```
cat journal.txt | grep erreur | wc -l
```

3 lignes → 2 lignes sélectionnées → résultat : 2

> remplace un fichier ; >> ajoute à la fin.

Le tube | transmet la sortie normale (stdout). Les erreurs (stderr) ne sont pas incluses par défaut.

Observer les lignes qui passent d'une commande à la suivante.

Observer les lignes qui passent d'une commande à la suivante.

---

4. Rechercher du texte — `grep`

`grep` filtre les lignes qui contiennent un motif :

```
grep "erreur" /var/log/syslog          # lignes contenant "erreur"
grep -i "erreur" /var/log/syslog       # insensible à la casse
grep -n "root" /etc/passwd             # avec numéro de ligne
grep -v "commentaire" fichier.conf     # lignes qui NE contiennent PAS le motif
grep -r "mot" /etc/                    # recherche récursive dans un dossier
```

`grep` combiné avec un tube

```
# Chercher les erreurs dans les logs
cat /var/log/syslog | grep -i "error"

# Compter les erreurs
grep -i "error" /var/log/syslog | wc -l

# Trouver les utilisateurs avec bash
cat /etc/passwd | grep "/bin/bash"
```

---

5. Quelques commandes de traitement utiles

Commande	Rôle
`wc -l`	Compte le nombre de lignes
`sort`	Trie les lignes par ordre alphabétique
`sort -n`	Trie numériquement
`uniq`	Supprime les lignes en double (après un `sort`)
`head -n 5`	Affiche les 5 premières lignes
`tail -n 5`	Affiche les 5 dernières lignes
`cut -d: -f1`	Coupe selon un séparateur et garde le 1er champ

```
# Afficher uniquement les noms d'utilisateurs depuis /etc/passwd
cut -d: -f1 /etc/passwd | sort

# Trouver les 3 plus gros fichiers dans /var/log
du -sh /var/log/* 2>/dev/null | sort -rh | head -3
```

Récapitulatif des commandes

Commande / Symbole	Description
`commande > fichier`	Redirige stdout vers un fichier (écrase)
`commande >> fichier`	Ajoute stdout à la fin d'un fichier
`commande 2> fichier`	Redirige stderr vers un fichier
`commande &> fichier`	Redirige stdout et stderr vers un fichier
`commande 2> /dev/null`	Jette les erreurs
`cmd1 | cmd2`	Connecte la sortie de cmd1 à l'entrée de cmd2
`grep "motif" fichier`	Filtre les lignes contenant le motif
`grep -i`	Insensible à la casse
`grep -v`	Lignes qui ne contiennent PAS le motif
`wc -l`	Compte les lignes
`sort`	Trie les lignes
`head -n N`	Affiche les N premières lignes
`tail -n N`	Affiche les N dernières lignes
`cut -d: -f1`	Découpe une ligne selon un séparateur

À retenir

- Chaque commande a trois flux : `stdin` (entrée), `stdout` (résultat), `stderr` (erreurs).
- `>` redirige stdout vers un fichier en **écrasant** ; `>>` **ajoute** à la fin.
- `2>` redirige uniquement les erreurs ; `&>` redirige les deux.
- `/dev/null` est le trou noir du système — tout ce qu'on y envoie est supprimé.
- Le **tube** `|` connecte la sortie d'une commande à l'entrée de la suivante.
- `grep "motif"` filtre les lignes contenant un texte — indispensable combiné aux tubes.
- `wc -l` compte les lignes, `sort` trie, `head`/`tail` extraient le début/la fin.

TP associé → `tp-09-redirections.md`

Chapitre 10 — Réseau et services

1. Vérifier la connectivité réseau

Afficher les interfaces réseau

```
ip address         # affiche toutes les interfaces et leurs adresses IP
ip -br address     # version courte et lisible
```

Exemple de sortie :

```
lo     UNKNOWN   127.0.0.1/8
enp0s3 UP        10.0.2.15/24
```

Interface	Description
`lo`	Loopback — interface locale (127.0.0.1)
`enp0s3`	Carte réseau Ethernet
`wlan0`	Carte réseau Wi-Fi

Tester la connectivité — `ping`

```
ping 8.8.8.8          # teste la connectivité vers un serveur Google
ping google.com        # teste aussi la résolution DNS
ping -c 4 google.com   # envoie 4 paquets et s'arrête
```

```
Ctrl + C    # pour arrêter un ping en cours
```

2. Se connecter à distance — SSH

SSH (*Secure Shell*) permet de se connecter à un autre ordinateur Linux **en ligne de commande**, de façon sécurisée et chiffrée.

> **Analogie** : SSH, c'est comme un téléphone sécurisé entre toi et un serveur distant. Tu tapes des commandes chez toi, elles s'exécutent là-bas.

```
ssh utilisateur@adresse_ip          # connexion standard
ssh stagiaire@192.168.1.10          # exemple avec IP locale
ssh -p 2222 stagiaire@192.168.1.10  # port non standard
```

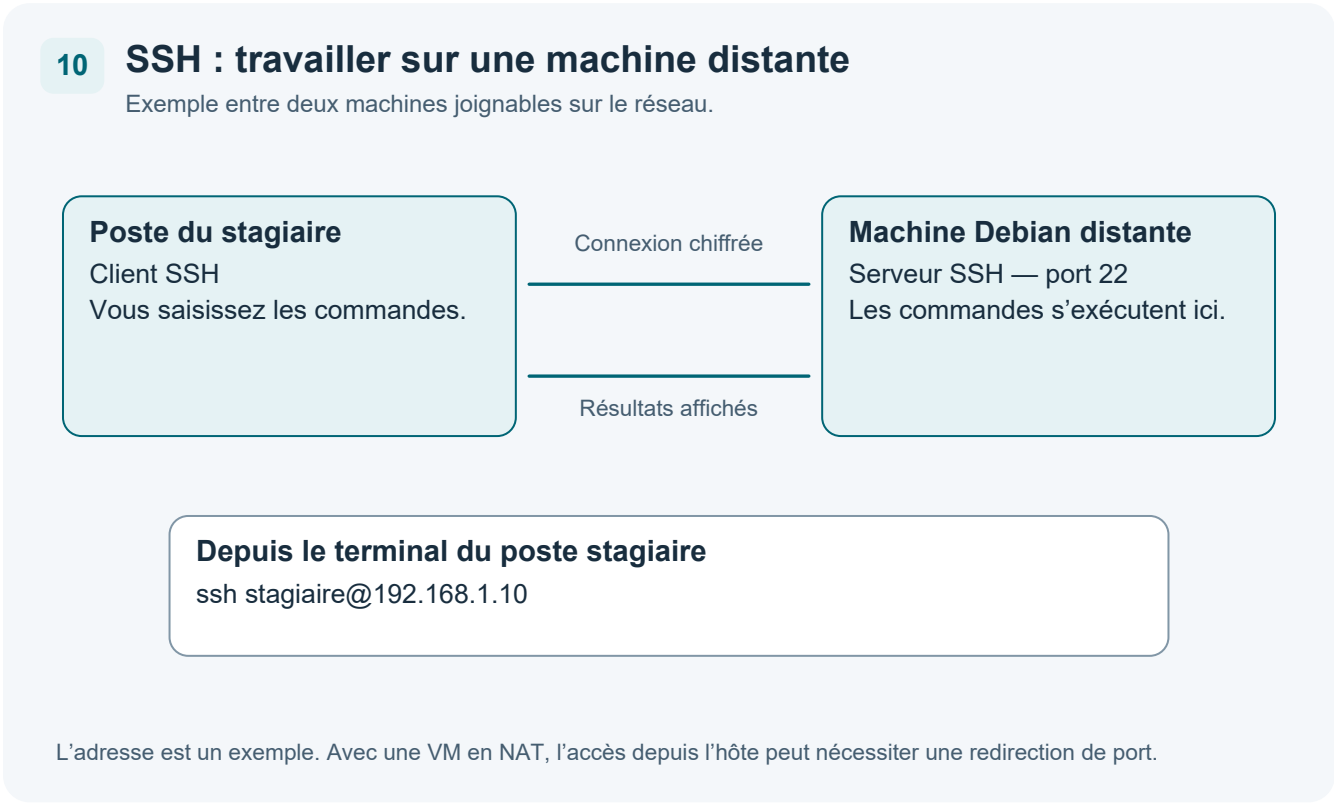
Pour quitter une session SSH :

```
exit
```

```
# ou Ctrl + D
```

SSH est l'outil principal pour administrer des serveurs Linux à distance. La quasi-totalité de l'administration système professionnelle se fait via SSH.

illustration-10



Les commandes SSH s’exécutent sur la machine distante.

Les commandes SSH s’exécutent sur la machine distante.

---

3. Résoudre des noms — DNS

Le **DNS** (*Domain Name System*) traduit les noms de domaine (google.com) en adresses IP (142.250.74.46).

```
# Vérifier la résolution DNS
host google.com
nslookup google.com

# Afficher les DNS configurés
cat /etc/resolv.conf
```

Le fichier `/etc/hosts` permet de définir des correspondances locales (sans DNS) :

```
cat /etc/hosts
# 127.0.0.1   localhost
# 192.168.1.10  mon-serveur
```

---

4. Les services (daemons)

Un **service** (ou *daemon*) est un programme qui tourne en arrière-plan en permanence. Exemples : serveur web, serveur SSH, base de données…

Linux moderne utilise **systemd** pour gérer les services.

Commandes essentielles

```
# État d'un service
sudo systemctl status ssh

# Démarrer / arrêter / redémarrer
sudo systemctl start ssh
sudo systemctl stop ssh
sudo systemctl restart ssh

# Activer / désactiver au démarrage
sudo systemctl enable ssh     # démarre automatiquement à chaque boot
sudo systemctl disable ssh    # ne démarre plus automatiquement

# Lister tous les services
systemctl list-units --type=service
systemctl list-units --type=service --state=running   # uniquement les actifs
```

Voir les logs d'un service

```
sudo journalctl -u ssh          # tous les logs du service ssh
sudo journalctl -u ssh -f       # en temps réel
sudo journalctl -u ssh --since "1 hour ago"  # depuis 1 heure
```

---

5. Informations réseau utiles

```
ss -tulnp          # liste les ports en écoute (remplace netstat)
ss -tulnp | grep 22   # vérifier si SSH est en écoute

ip route           # table de routage (passerelle par défaut)
```

Récapitulatif des commandes

Commande	Description
`ip address`	Affiche les interfaces réseau et leurs adresses IP
`ip -br address`	Version courte et lisible
`ping adresse`	Teste la connectivité réseau
`ping -c 4 adresse`	Envoie 4 paquets et s'arrête
`ssh user@ip`	Connexion SSH à distance
`exit`	Quitter une session SSH
`host domaine`	Résolution DNS d'un nom de domaine
`cat /etc/hosts`	Voir les correspondances DNS locales
`sudo systemctl status service`	État d'un service
`sudo systemctl start service`	Démarrer un service
`sudo systemctl stop service`	Arrêter un service
`sudo systemctl restart service`	Redémarrer un service
`sudo systemctl enable service`	Activer au démarrage
`sudo systemctl disable service`	Désactiver au démarrage
`sudo journalctl -u service`	Voir les logs d'un service
`ss -tulnp`	Liste les ports en écoute

À retenir

- `ip address` (ou `ip -br a`) affiche les interfaces réseau et leurs adresses IP.
- `ping adresse` teste la connectivité réseau — `Ctrl+C` pour arrêter.
- **SSH** permet de se connecter à distance de façon sécurisée : `ssh utilisateur@ip`.
- Le fichier `/etc/hosts` définit des correspondances nom ↔ IP locales (sans DNS).
- Un **service** (daemon) est un programme tournant en arrière-plan — géré par **systemd**.
- `systemctl start/stop/restart` démarre/arrête/redémarre un service.
- `systemctl enable/disable` configure le démarrage automatique au boot.
- `journalctl -u service` consulte les logs d'un service.

Chapitre 01 — Le shell et son environnement

Objectif Te repérer dans le terminal, lancer une commande et créer ton espace de travail. Distinguer les principaux shells et observer un sous-shell. Tu n'as encore aucun script à écrire.

1. Où taper les commandes ?

Depuis **PowerShell sur Windows**, connecte-toi à ta VM :

```
ssh stagiaire@192.168.84.10
```

La suite se déroule dans le terminal **Debian**, après la connexion. Adapte l'adresse si ton formateur t'en donne une autre. Le mot de passe ne s'affiche pas pendant la saisie.

Une ligne semblable à celle-ci attend ta commande :

```
stagiaire@debian:~$
```

Ne recopie pas cette invite ni le signe `$` : tape seulement la commande.



Du terminal à la commande exécutée

Le **terminal** est la fenêtre. Le **shell** est le programme qui interprète tes commandes. Ici, ce shell est **Bash**. SSH transporte tes échanges avec la VM.

2. Plusieurs shells, un même rôle

Linux peut utiliser plusieurs interpréteurs de commandes :

Nom	Repère pour débuter
`sh`	Nom historique et interface de référence pour les scripts portables. Sur notre Debian, `/bin/sh` lance **dash**.
`bash`	Shell utilisé dans cette formation. Il propose des fonctions supplémentaires, comme les tests `[[ ... ]]`.

Nom	Repère pour débuter
`zsh`, `fish`	Autres shells, avec leurs propres fonctions et réglages. Ils ne sont pas nécessaires pour ce parcours.

Vérifie les shells déclarés sur la VM, le programme utilisé pour `sh` et ton shell actuel :

```
cat /etc/shells
readlink -f /bin/sh
ps -p $$ -o comm=
```

- `/etc/shells` liste les shells de connexion déclarés ; la liste varie selon la machine.
- `readlink -f` suit le lien jusqu'au programme réel. Ici, le résultat est `/usr/bin/dash`.
- `ps` affiche un processus ; `-p $$` sélectionne le shell actuel, et `-o comm=` affiche son nom sans en-tête. Ici : `bash`.

La variable `$SHELL` indique normalement ton shell de connexion configuré. Elle ne permet pas de savoir avec certitude quel interpréteur exécute la commande actuelle : lancer `sh` depuis Bash ne la met pas automatiquement à jour.

Un shell est **interactif** quand il attend tes commandes, comme après notre connexion SSH. Il est **non interactif** quand il exécute un script ou une commande fournie avec `-c`. Un shell **de connexion** initialise aussi l'environnement de la session : c'est une autre distinction. Pour l'instant, retiens seulement que ces modes peuvent lire des fichiers de configuration différents ; `~/.bashrc` sert notamment aux réglages des sessions Bash interactives.

`sh` et Bash ne sont pas interchangeables Nos futurs scripts utiliseront Bash. Les lancer avec `sh script.sh` peut échouer s'ils emploient des fonctions propres à Bash. Nous apprendrons à choisir l'interpréteur avec le **shebang**, au chapitre 03.

3. Un shell peut en lancer un autre

Après SSH, tu travailles dans un shell. Si tu tapes `bash`, il démarre un **nouveau Bash enfant**. La commande `exit` termine ce Bash et te ramène dans le parent. Un nouvel `exit` dans le parent ferme alors la connexion SSH.

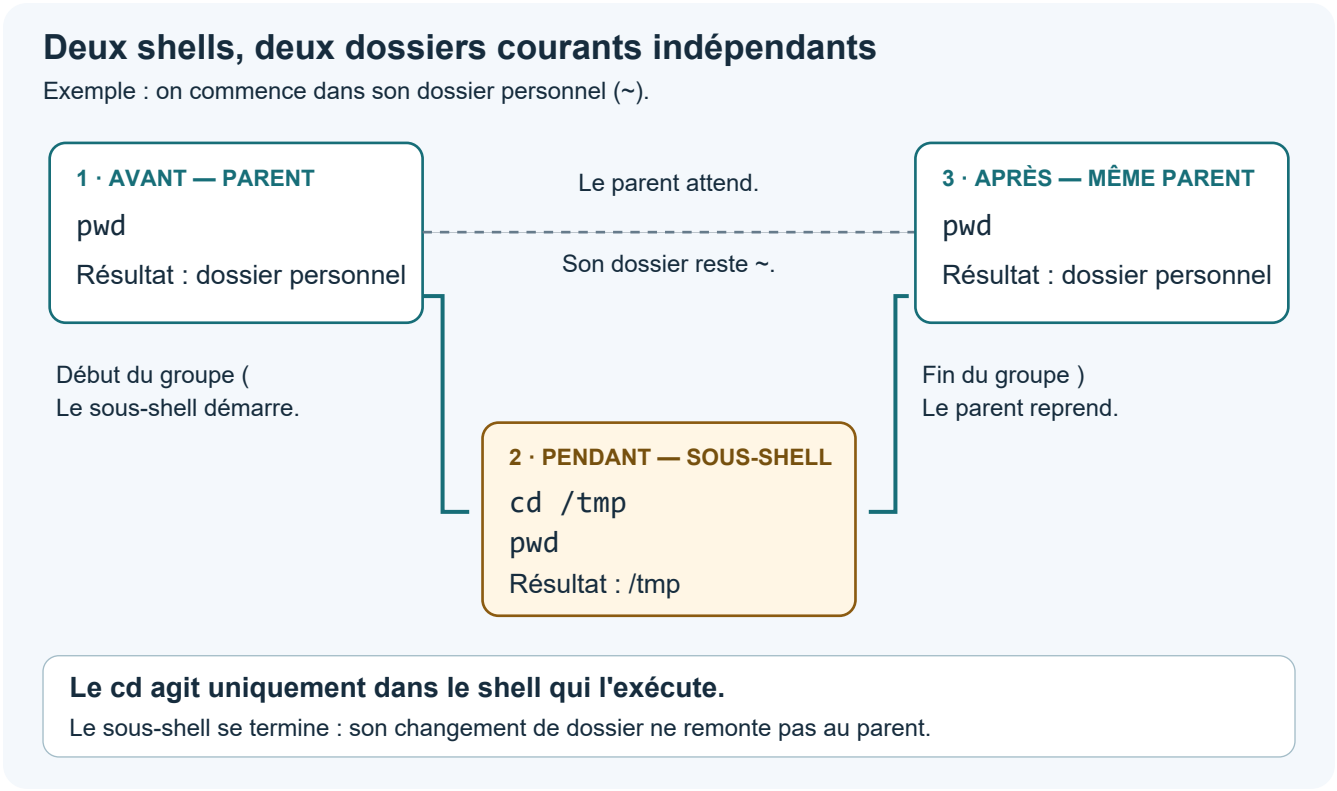
Pour une seule commande, `bash -c 'commande'` démarre un Bash enfant, exécute le texte fourni puis se termine :

```
bash -c 'echo "Bonjour depuis le Bash enfant"'
echo "De retour dans le shell parent"
```

Le shell enfant est un processus distinct. Il ne peut pas changer le dossier courant de son parent.

Les parenthèses permettent aussi d'exécuter un groupe de commandes dans un **sous-shell**, avec une copie de l'environnement du shell courant :

Imagine une cabine qui reçoit une copie de ton poste de travail. Elle peut changer son propre dossier sans déplacer ton poste. **Le parent attend pendant que la cabine travaille, puis reprend quand elle se ferme.** On reste sur la même VM : aucune nouvelle connexion SSH n'est ouverte.



Avant, pendant et après : le sous-shell change son dossier vers /tmp, le parent conserve son dossier personnel

Prédis avant de lancer : le dernier `pwd` affichera-t-il `/tmp` ou ton dossier personnel ? Dans cet exemple, `cd ~` place d'abord le parent dans ton dossier personnel.

```
cd ~
pwd
(
    cd /tmp
    pwd
)
pwd
```

Vérifie les trois résultats :

```
/home/stagiaire
/tmp
/home/stagiaire
```

Sur notre VM, le dossier personnel est `/home/stagiaire` ; adapte ce nom si tu utilises un autre compte. Le premier et le dernier résultat sont identiques. Le `cd /tmp` a agi seulement dans le sous-shell.

À ton tour : remplace `/tmp` par `/etc` dans le groupe. Prédis les trois lignes, puis teste. Seule la ligne du milieu doit changer.

Les variables suivront une logique similaire : nous verrons leur transmission avec `export` au chapitre 02.

4. Une commande, ses options et ses arguments

```
ls -l /tmp
```

Élément	Ce qu'il demande
`ls`	Lister un dossier
`-l`	Afficher les détails : c'est une option
`/tmp`	Choisir le dossier à lister : c'est un argument

Une commande se termine quand tu appuies sur **Entrée**. Bash distingue les majuscules et les minuscules : `ls` et `LS` sont deux noms différents.

5. Se repérer avant d'agir

```
whoami
hostname
pwd
```

- `whoami` : ton compte ; ici, normalement `stagiaire`.
- `hostname` : le nom de la machine distante.
- `pwd` : le dossier où tu travailles ; au départ, normalement `/home/stagiaire`.

Crée un espace réservé à cette formation :

```
mkdir -p ~/atelier-bash/decouverte
cd ~/atelier-bash/decouverte
pwd
```

`~` désigne ton dossier personnel. `-p` crée aussi les parents manquants et accepte qu'un dossier existe déjà. `cd` change le dossier courant.

Chemin	Signification
`/home/stagiaire`	Chemin absolu, depuis la racine `/`
`decouverte`	Chemin relatif au dossier courant
`.`	Dossier courant
`..`	Dossier parent
`~`	Dossier personnel

```
cd ..
pwd
cd decouverte
```

Tu reviens maintenant dans `~/atelier-bash/decouverte`.

6. Écrire puis lire un fichier

```
echo "Station Orion ouverte" > journal.txt
cat journal.txt
```

Résultat :

```
Station Orion ouverte
```

`echo` affiche le texte puis ajoute automatiquement un retour à la ligne. Le signe `>` dirige ce texte vers un fichier : **il crée le fichier ou remplace son contenu**.

Pour ajouter une ligne :

```
echo "Radio disponible" >> journal.txt
cat journal.txt
```

`>>` ajoute à la fin. Il ne remplace pas les lignes déjà présentes.

> Avant d'utiliser `>` Vérifie le nom du fichier et ton dossier avec `pwd`. Dans le cours, nous écrivons seulement dans notre espace de travail.

7. Passer un résultat à une autre commande

```
cat journal.txt | grep Radio
```

Le tube `|` transmet la sortie de `cat` à `grep`. `grep` garde les lignes contenant `Radio`. Le fichier d'origine reste intact.

On peut aussi demander directement à `grep` de lire le fichier :

```
grep Radio journal.txt
```

Ces deux exemples affichent `Radio disponible`. Le premier montre comment relier deux commandes ; le second est plus court pour ce besoin précis.

8. Les raccourcis utiles

Geste	Effet
Tab	Complète un nom ; deux appuis peuvent proposer les choix
Flèche haut	Rappelle une commande précédente
Ctrl+C	Interrompt la commande en cours
Ctrl+L	Nettoie l'affichage sans effacer les fichiers
`exit`	Ferme la session SSH

L'aide de `ls` se consulte avec `ls --help`. Pour une commande intégrée à Bash, comme `cd`, utilise `help cd`. Le lecteur `man ls` se quitte avec **q**.

9. Pour aller plus loin : un alias temporaire

Un alias donne un raccourci à une commande. Cet exemple agit seulement dans ton shell actuel :

```
alias ll='ls -l'
ll
unalias ll
```

Nous ne modifions pas encore les fichiers de configuration du shell. Plus tard, `~/.bashrc` permettra de conserver des réglages pour les shells Bash interactifs qui le lisent.

À retenir

- Tu tapes dans un terminal ; Bash interprète les commandes sur Debian.
- Sur cette VM, `sh` lance dash ; nos scripts seront écrits pour Bash.
- Le `cd` effectué dans un sous-shell ne change pas le dossier du parent.
- `pwd` permet de vérifier où tu es avant d'écrire un fichier.
- `>` remplace ; `>>` ajoute ; `|` transmet un résultat.

À toi : quelle différence entre `cd ..` et `cd ~` ? Que contient le journal après deux ajouts ? Pourquoi le `cd /tmp` entre parenthèses ne te déplace-t-il pas définitivement ?

Passer au TP

Chapitre 02 — Les variables

Objectif Mémoriser une valeur, l'afficher dans une phrase et demander une réponse à l'utilisateur.

1. Une variable donne un nom à une valeur

```
prenom="Mila"
echo "Bonjour $prenom"
```

Résultat : `Bonjour Mila`.

La première ligne **affecte** une valeur : Bash range `Mila` et retient le nom `prenom`. La seconde **lit** cette valeur avec `$prenom` : Bash remplace le nom par la valeur, puis `echo` affiche la phrase obtenue.

Une valeur, un nom, une utilisation



De l'affectation à l'affichage

Écriture	Ce qu'elle signifie
`prenom="Mila"`	Ranger `Mila` dans la variable `prenom`
`$prenom`	Demander la valeur rangée sous ce nom
`"$prenom"`	Utiliser cette valeur comme un seul argument

Le `$` est la marque de la **lecture**. On ne l'écrit jamais au moment d'affecter.

Aucun espace autour de `=` `prenom="Mila"` fonctionne. `prenom = "Mila"` échoue : Bash y voit une commande nommée `prenom`, à laquelle on passerait les arguments `=` et `Mila`.

```
prenom : commande introuvable
```

C'est l'erreur la plus fréquente du chapitre. Le `=` se colle des deux côtés.

Choisis des noms lisibles : `prenom`, `nom_station`, `nombre_messages`. Un nom ne commence pas par un chiffre. `prenom` et `PRENOM` sont deux variables différentes.

2. Les guillemets doubles gardent la valeur en un seul morceau

Après avoir remplacé `$station` par sa valeur, Bash **découpe le résultat en mots** à chaque espace — sauf si la lecture est entre guillemets doubles.

```
station="Orion    Nord"
echo $station
echo "$station"
```

Résultats, dans cet ordre :

```
Orion Nord
Orion    Nord
```

Sans guillemets, Bash a coupé la valeur en deux arguments, `Orion` et `Nord`, et `echo` les a recollés avec **un seul** espace. Les espaces d'origine ont disparu : c'est la preuve visible que la valeur a été découpée.

Avec guillemets, `echo` reçoit **un seul** argument, qui contient la valeur exacte.

Tant qu'on affiche du texte, le dégât reste cosmétique. Il devient réel dès qu'on manipule un fichier :

```
fichier="rapport final.txt"
cat $fichier      # cat cherche DEUX fichiers : « rapport » puis « final.txt »
cat "$fichier"    # cat cherche LE fichier : « rapport final.txt »
```

> Habitude à prendre Quand tu utilises une variable comme argument, écris `"$variable"`. Les guillemets ne coûtent rien quand la valeur n'a pas d'espace, et sauvent la commande quand elle en a.

3. Guillemets simples ou guillemets doubles ?

Les deux protègent les espaces. Ils se distinguent sur un seul point : le `$`.

```
station="Orion Nord"
echo "$station"
echo '$station'
```

Résultats, dans cet ordre :

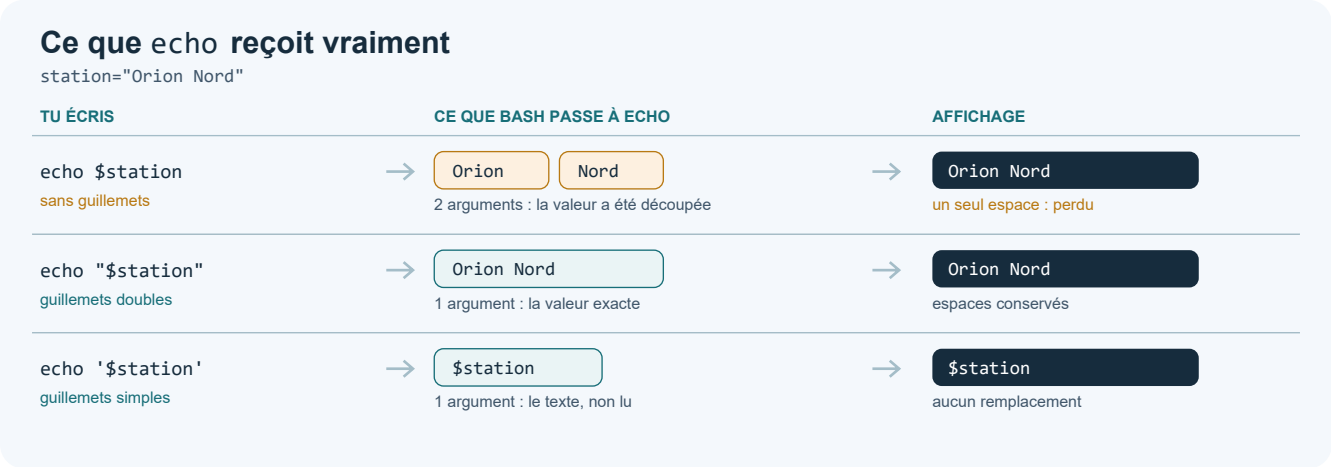
```
Orion Nord
$station
```

- Les **guillemets doubles** laissent Bash lire la variable, tout en gardant sa valeur groupée.
- Les **guillemets simples** conservent le texte tel quel : aucun `$` n'est interprété.

Tu veux	Écris
La valeur de la variable	`"$station"`
Le texte `$station` lui-même	`'$station'`
Un texte à transmettre tel quel à un autre programme	`'echo $station'`

La troisième ligne servira au §9 : elle permet d'envoyer un `$` à un autre shell sans que le tien y touche au passage.

Les trois écritures des §2 et §3, côte à côte :



Ce que echo reçoit vraiment selon les guillemets

4. Les accolades : délimiter le nom, prévoir un défaut

Délimiter le nom

```
mission="orion"
fichier="${mission}_rapport.txt"
echo "$fichier"
```

Résultat : `orion_rapport.txt`.

Les accolades disent où le nom s'arrête. Sans elles, `"$mission_rapport.txt"` fait chercher à Bash une variable nommée `mission_rapport` — le souligné fait partie des caractères autorisés dans un nom. Cette variable n'existe pas, donc Bash la remplace par du vide : il ne reste que `.txt`.

Prévoir une valeur par défaut

```
surnom=""
echo "Nom affiche : ${surnom:-visiteur}"
surnom="Mila"
echo "Nom affiche : ${surnom:-visiteur}"
```

Résultats, dans cet ordre :

```
Nom affiche : visiteur
Nom affiche : Mila
```

`${surnom:-visiteur}` fournit `visiteur` **seulement** si `surnom` est vide ou absente. La variable n'est pas remplie pour autant : après le premier affichage, `surnom` est toujours vide.

5. Poser une question avec read

```
read -p 'Ton prenom : ' prenom
echo "Bienvenue $prenom"
```

- `read` attend une ligne et la range dans `prenom`.

- `-p` affiche la question dans le terminal.

Tape une réponse, puis appuie sur **Entrée**. Le nom après la question est celui de la variable : on écrit `prenom`, sans `$`, parce que c'est une affectation.

Avec une seule variable, `read` range **toute la ligne**, espaces compris : une réponse comme `Mila Keller` arrive entière dans `prenom`. Raison de plus pour la relire avec `"$prenom"`.

Ces lignes s'exécutent dans ton terminal Bash. Le programme attend réellement ta saisie : ce n'est pas un blocage.

6. Garder le résultat d'une commande

```
machine=$(hostname)
jour=$(date +%F)
echo "Machine : $machine — date : $jour"
```

`$(...)` exécute la commande et récupère sa sortie. Ici, Bash garde le nom de la machine et la date au format année-mois-jour. La valeur de la date dépend du jour où tu exécutes l'exemple.

Ne confonds pas : `"$machine"` lit une variable, `$(hostname)` lance une commande.

7. Calculer avec des nombres entiers

```
batteries=4
energie_par_batterie=25
energie_totale=$((batteries * energie_par_batterie))
echo "Energie disponible : $energie_totale unites"
```

Résultat : `Energie disponible : 100 unites`.

Une variable Bash contient toujours du **texte** : `batteries` contient le texte `4`, pas un nombre. `$((...))` demande justement à Bash de lire ce texte comme un nombre et de calculer avec. C'est la raison d'être des doubles parenthèses.

À l'intérieur, le `$` de lecture devient facultatif : `$((batteries * 25))` et `$(($batteries * 25))` donnent le même résultat. La forme sans `$` est la plus courante.

Opérateur	Calcul
`+`, `-`	Addition, soustraction
`*`	Multiplication
`/`	Division entière : `7 / 2` donne `3`
`%`	Reste : `7 % 2` donne `1`

Nous utilisons ici des valeurs fixées dans le cours. Avant de calculer avec une saisie libre, il faudra vérifier qu'elle contient bien un nombre : ce sera un défi du chapitre 04.

8. Les variables déjà définies par le système

Ton shell contient déjà des variables que tu n'as pas créées :

```
echo "Dossier personnel : $HOME"
echo "Recherche des commandes : $PATH"
```

`HOME` désigne ton dossier personnel. `PATH` liste les dossiers où le shell recherche les commandes quand leur nom ne contient pas de `/`. La commande `env` affiche toutes les variables de cette famille ; nous y revenons au §9.

Elles se lisent exactement comme les tiennes, avec `$`. Par convention, ces variables du système s'écrivent en MAJUSCULES : garde les minuscules pour les tiennes, tu éviteras d'en écraser une par accident.

9. Transmettre une variable à un autre programme : export

Une variable ordinaire n'existe que dans le shell qui l'a créée. `export` la rend disponible aux programmes lancés **ensuite** par ce shell.

```
export NOM_STATION="Orion"
bash -c 'echo "Station recue : [$NOM_STATION]"'
```

Résultat : `Station recue : [Orion]`. Le second Bash est un processus enfant, comme au chapitre 01 ; il reçoit `NOM_STATION` grâce à `export`.

La contre-épreuve, sans `export` :

```
PAS_EXPORTEE="Vega"
bash -c 'echo "Non exportee : [$PAS_EXPORTEE]"'
```

Résultat : `Non exportee : []`. Les crochets sont vides : l'enfant n'a jamais reçu la variable. C'est précisément ce que `export` corrige.

> Les guillemets simples de `bash -c '...'` sont indispensables Ils empêchent **ton** shell de remplacer `$PAS_EXPORTEE` avant de lancer l'enfant. Avec des guillemets doubles, tu verrais s'afficher `[Vega]` — mais la valeur viendrait de ton propre shell, pas de l'enfant : la démonstration serait fausse alors que le résultat a l'air juste. C'est le §3 appliqué à un cas réel.

Voir quelles variables existent

Deux commandes listent les variables de ton shell, et la différence entre les deux est précisément celle de ce paragraphe :

Commande	Ce qu'elle liste
`env`	Uniquement les variables **exportées**, c'est-à-dire l'environnement
`set`	**Toutes** les variables du shell, exportées ou non

`set` donne donc toujours une liste plus longue que `env` : elle contient en plus les variables ordinaires, les tiennes. Essayons avec une de chaque :

```
locale_seule="Vega"
export EXPORTEE="Orion"
```

Les deux listes défilent sur des dizaines de lignes : on cherche un nom précis avec `grep`, le filtre vu au chapitre 01.

```
env | grep EXPORTEE         # trouvée : elle est dans l'environnement
env | grep locale_seule     # rien : elle n'y est pas
set | grep locale_seule     # trouvée : elle existe bien dans ton shell
```

Une variable ordinaire existe donc pour ton shell, mais pas pour l'environnement qu'il transmet à ses enfants. C'est la même constatation qu'avec `bash -c`, obtenue cette fois sans lancer de processus.

Portée et durée de vie. Une variable disparaît avec le processus qui la contient : rien de ce que tu définis ici ne survit à la fermeture du terminal. `export` transmet vers les enfants, jamais vers le parent — aucun script ne peut donc modifier les variables de ton shell. Ces exemples ne touchent à aucun fichier de configuration.

Pour aller plus loin : printf

Pour les messages simples de nos premiers exercices, `echo "Bonjour $prenom"` suffit. `printf` devient utile quand on veut maîtriser le format d'une sortie :

```
printf 'Bonjour %s\n' "$prenom"
```

- Le premier argument est le **format** : `Bonjour %s\n`.
- `%s` reçoit le texte de l'argument suivant, ici `"$prenom"`.
- `\n` ajoute un retour à la ligne. `printf` ne l'ajoute pas automatiquement.

Cette commande donne ici la même phrase qu'avec `echo`. Mais `printf` montre aussi où commence et où finit chaque argument, ce qui rend le §2 enfin lisible :

```
station="Orion Nord"
printf '[%s]\n' "$station"
printf '[%s]\n' $station
```

```
[Orion Nord]
[Orion]
[Nord]
```

Un argument d'un côté, deux de l'autre : le découpage en mots devient visible.

Enfin, `printf '%s\n' "$variable"` traite comme du texte une valeur telle que `-n` ou contenant des antislashs, sans la confondre avec une option ou un échappement de `echo`.

> Un repère pour la suite Utilise `echo` pour une phrase simple. Utilise `printf` pour un format précis, plusieurs lignes ou des données dont tu ne maîtrises pas le contenu. Garde le format fixe et passe les variables dans les arguments suivants.

À retenir

- Affecter : `prenom="Mila"`, sans espace autour du `=`. Lire : `"$prenom"`.
- Les guillemets **doubles** laissent lire le `$` et gardent la valeur en un seul argument ; les **simples** conservent le texte tel quel.
- `${nom}` délimite le nom ; `${nom:-defaut}` fournit une valeur de secours sans remplir la variable.
- `read` récupère une saisie, `$(...)` la sortie d'une commande, `$((...))` le résultat d'un calcul entier.
- Une variable contient toujours du texte ; `$((...))` demande à Bash de le lire comme un nombre.
- `export` transmet une variable aux programmes lancés ensuite ; sans lui, l'enfant reçoit une valeur vide.
- `env` liste les variables exportées, `set` les liste toutes : la différence, ce sont les tiennes.
- `printf` prend le relais de `echo` quand le format compte.

À toi : que produit `printf '%s\n' '$prenom'` ? Et si tu remplaces les guillemets simples par des doubles ? Que contient `fichier` après `mission="orion"` puis `fichier="$mission_rapport.txt"`, et pourquoi ?

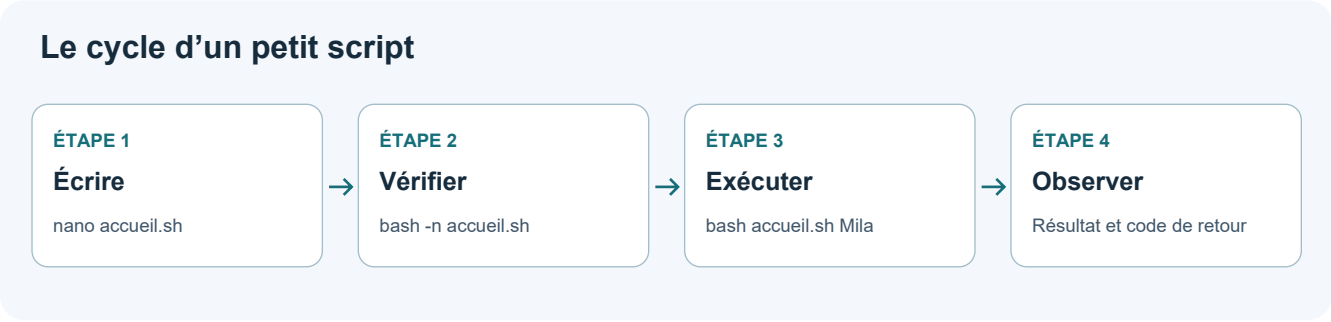
Passer au TP

Chapitre 03 — Écrire son premier script

Objectif Transformer quelques commandes en un programme que tu peux relancer, lire et vérifier.

1. Un script est un fichier texte

Un script Bash contient les commandes que tu aurais pu taper dans le terminal. Bash les lit dans l'ordre. Tu peux ainsi refaire la même tâche sans recopier les commandes.



Écrire, vérifier puis exécuter

Commence dans un dossier de travail :

```
mkdir -p ~/atelier-bash/scripts
cd ~/atelier-bash/scripts
```

2. Créer le fichier avec nano

Ouvre l'éditeur :

interactif: editeur

```
nano accueil.sh
```

Dans l'éditeur, saisis le contenu suivant. Ne recopie pas les marques de bloc de code.

fichier: accueil.sh

```
#!/usr/bin/env bash

# Un nom peut etre fourni au lancement.
prenom="${1:-visiteur}"
station="Orion"

echo "Bienvenue $prenom sur $station."
```

Ligne	Rôle
`#!/usr/bin/env bash`	Demande d'utiliser Bash pour l'exécution directe

Ligne	Rôle
`# Un nom…`	Commentaire : Bash l'ignore, il s'adresse à qui lit le fichier
`prenom="${1:-visiteur}"`	Prend le premier argument fourni au script, ou `visiteur` s'il n'y en a pas
`station="Orion"`	Range une valeur fixe
`echo ...`	Produit le message

La troisième ligne est la plus chargée : `$1` désigne le premier argument et `${1:-visiteur}` fournit une valeur de secours, comme le `${nom:-defaut}` du chapitre 02. Le §5 y revient en détail — pour l'instant, recopie-la telle quelle.

La première ligne s'appelle le **shebang**. Nous écrivons `#!/usr/bin/env bash` plutôt que `#!/bin/bash` : `env` va chercher Bash là où il est réellement installé, ce qui rend le script utilisable sur des systèmes où le chemin diffère.

Dans nano : **Ctrl+O**, puis **Entrée** pour enregistrer ; **Ctrl+X** pour quitter. Les indications `^O` et `^X` de nano signifient « Ctrl+O » et « Ctrl+X ».

Fichier créé sous Windows Si tu vois une erreur mentionnant `bash\r`, le fichier a été enregistré avec des fins de ligne Windows (CRLF). Un script Linux doit être enregistré en **LF**. En travaillant avec nano dans Debian, tu évites ce problème.

3. Vérifier avant de lancer

```
bash -n accueil.sh
```

`-n` vérifie la syntaxe **sans exécuter** les commandes. Si rien ne s'affiche et que la commande réussit, Bash n'a pas trouvé d'erreur de syntaxe. Cela ne garantit pas que le programme fait ce que tu souhaites : il faut aussi le tester.

Lance-le ensuite en indiquant l'interpréteur :

```
bash accueil.sh Mila
```

Résultat : `Bienvenue Mila sur Orion.`. Dans ce mode, le fichier n'a pas besoin d'être exécutable.

4. Exécuter directement le script

```
chmod +x accueil.sh
./accueil.sh Mila
```

- `chmod +x` ajoute le droit d'exécution.
- `./accueil.sh` désigne le fichier dans le dossier courant.
- `Mila` est un argument transmis au script.

Le `./` n'est pas décoratif : le dossier courant n'est généralement pas dans `PATH`. Pour un script stocké ailleurs, indique son chemin.

Noter aussi ce qui **n'est pas** nécessaire : aucun `sudo` ici. Tu es propriétaire de ton dossier personnel, donc tu peux y créer un fichier, le rendre exécutable et le lancer sans droits administrateur. Un script qui réclame `sudo` pour travailler dans `~` est le signe qu'il se trompe de dossier.

5. Les arguments : des valeurs fournies au départ

Dans le script, `$1` est le premier argument. `${1:-visiteur}` donne une valeur par défaut si ce premier argument est absent ou vide.

```
./accueil.sh
./accueil.sh "Mila Vega"
```

Résultats :

```
Bienvenue visiteur sur Orion.
Bienvenue Mila Vega sur Orion.
```

Les guillemets autour de `Mila Vega` conservent un seul argument.

Variable spéciale	Signification
`$0`	Nom utilisé pour lancer le script
`$1`, `$2`	Premier et deuxième arguments
`$#`	Nombre d'arguments
`"$@"`	Tous les arguments, en conservant leurs limites

Pour observer les arguments, crée un second fichier :

fichier: arguments.sh

```
#!/usr/bin/env bash
echo "Nombre : $#"
printf 'Argument : %s\n' "$@"
```

```
bash arguments.sh "Mila Vega" pilote
```

```
Nombre : 2
Argument : Mila Vega
Argument : pilote
```

`"$@"` ne recoupe pas `Mila Vega` en deux mots : c'est bien un seul argument.

Une ligne de format, deux lignes affichées : `printf` **réapplique son format tant qu'il reste des arguments à consommer**. C'est le seul endroit de ce chapitre où `printf` rend un service qu'`echo` ne sait pas rendre — `echo "Argument : $@"` n'afficherait qu'une seule ligne, avec les deux valeurs à la suite. Partout ailleurs, `echo` suffit.

6. Le code de retour

Une commande fournit un **code de retour**. `0` signifie qu'elle a réussi selon ses propres règles. Une valeur différente de zéro indique un autre résultat : erreur, absence de correspondance pour `grep`, etc.

```
bash accueil.sh Mila
code=$?
echo "Code : $code"
```

`$?` donne le code de la **dernière commande**. Ici, le script réussit et le code est `0`. Il faut le lire tout de suite, avant qu'une autre commande le remplace.

Observe un échec sans créer de fichier :

```
ls dossier-inexistant-orion 2>/dev/null
code=$?
echo "Code apres echec : $code"
```

Le code est différent de zéro. `2>/dev/null` masque le message d'erreur ; il ne transforme pas l'échec en réussite. Nous verrons comment réagir au résultat dans le chapitre suivant.

7. Comprendre un comportement inattendu

```
bash -x accueil.sh Mila
```

`-x` affiche les commandes après les expansions. Cela aide à comprendre les valeurs réellement utilisées. Les lignes de trace et le résultat normal peuvent apparaître ensemble.

> Trace et données personnelles Une trace peut afficher les valeurs des variables. Utilise les données fictives du cours avant de partager une capture.

À retenir

- Le shebang choisit l'interpréteur lors de l'exécution directe ; nous utilisons `#!/usr/bin/env bash`.
- `bash -n` vérifie la syntaxe ; les essais vérifient le comportement ; `bash -x` montre les valeurs réellement utilisées.
- `chmod +x` donne le droit d'exécution, `./` indique le fichier du dossier courant : les deux sont nécessaires.
- Les arguments se lisent avec `$1`, `$#` et `"$@"` ; `${1:-defaut}` prévoit le cas où l'argument est absent.
- Protège les arguments avec des guillemets.
- `$?` donne le code de retour de la dernière commande : `0` pour une réussite, autre chose sinon. Lis-le tout de suite.
- Un script n'a pas besoin de `sudo` pour travailler dans ton dossier personnel.

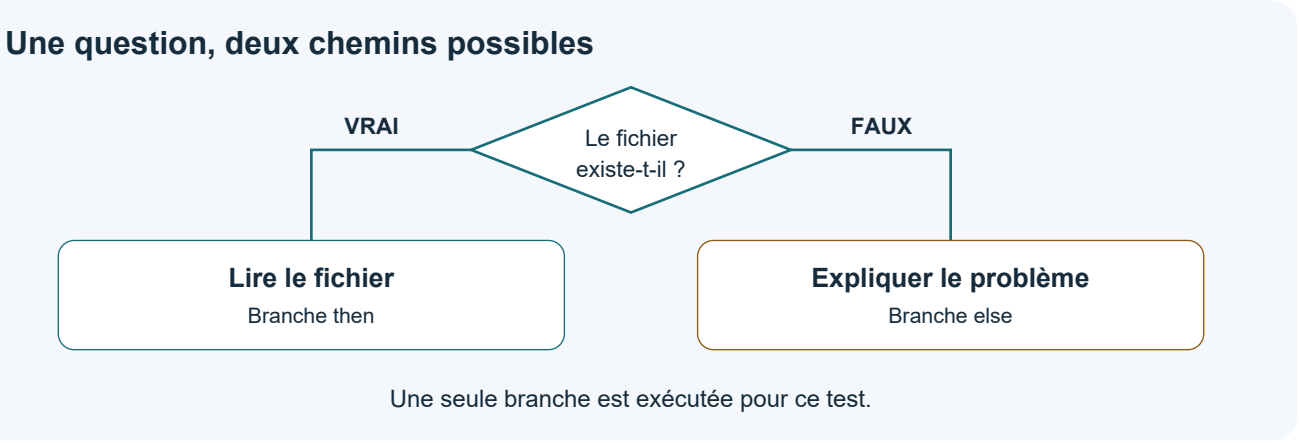
À toi : combien d'arguments reçoit `bash arguments.sh "poste radio" Orion` ? Pourquoi `chmod +x` ne suffit-il pas à lancer un fichier ? Que vaut `$?` juste après une commande qui a échoué ?

Passer au TP

Chapitre 04 — Les tests et les conditions

Objectif Faire prendre une décision à ton script : continuer, choisir une action ou expliquer pourquoi il s'arrête.

1. Une question avec deux chemins



Un test choisit entre deux actions

`if` lance une commande, lit son **code de retour** et choisit un chemin en conséquence.

```
destination="Orion"
if [[ "$destination" == "Orion" ]]; then
    echo 'Bonne destination'
else
    echo 'Verifier le billet'
fi
```

Résultat : `Bonne destination`. Avec `destination="Vega"`, tu prendrais l'autre chemin.

`then` ouvre le premier bloc, `else` introduit le second, `fi` ferme le `if`. L'indentation de quatre espaces rend les branches visibles.

Ici, `0` veut dire vrai Souviens-toi du chapitre 03 : une commande qui réussit renvoie le code `0`. `if` ouvre le bloc `then` quand la commande **réussit**, donc quand elle renvoie `0`. C'est l'inverse de ce que tu as peut-être vu ailleurs, où `0` signifie « faux ».

`[[ ... ]]` est une commande comme une autre : elle n'affiche rien, elle renvoie seulement un code.

```
[[ "Orion" == "Orion" ]]; echo "$?"   # 0 : le test réussit
[[ "Orion" == "Vega" ]];  echo "$?"   # 1 : le test échoue
```

C'est cette mécanique qui permettra, au §5, d'utiliser n'importe quelle commande comme test.

Dans ce cours, nous utilisons `[[ ... ]]`, une syntaxe de **Bash**. Garde les espaces après `[[` et avant `]]`. Tu croiseras souvent `[ ... ]`, avec un seul crochet : c'est la forme historique, commune à tous les shells. Elle fait le même travail avec davantage de pièges — en Bash, garde `[[ ... ]]`.

2. Comparer du texte

Reprenons la même variable pour poser les autres questions possibles :

Test	Question posée
`[[ "$destination" == "Orion" ]]`	La valeur est-elle exactement `Orion` ?
`[[ "$destination" != "Orion" ]]`	Est-elle différente d'`Orion` ?
`[[ -z "$destination" ]]`	La valeur est-elle vide ?
`[[ -n "$destination" ]]`	Contient-elle au moins un caractère ?

`-z` sert surtout juste après un `read` : il repère une réponse laissée vide.

```
read -p 'Destination : ' destination
if [[ -z "$destination" ]]; then
    echo 'Aucune destination saisie'
else
    echo "Cap sur $destination"
fi
```

Essaie les deux cas : tape un nom, puis relance en appuyant directement sur **Entrée**.

Le signe `=` d'une affectation n'est pas un test. `destination="Orion"` change la variable ; le test, lui, vérifie sa valeur sans la modifier.

3. Comparer des entiers

Pourquoi un second jeu d'opérateurs ? Parce qu'une variable Bash contient toujours du **texte**. Il n'existe pas de variable « nombre » d'un côté et « texte » de l'autre : c'est l'**opérateur** qui décide comment cette valeur est lue.

Opérateur	Signification
`-eq`, `-ne`	Égal, différent
`-lt`, `-le`	Inférieur, inférieur ou égal
`-gt`, `-ge`	Supérieur, supérieur ou égal

```
energie=30
if [[ "$energie" -ge 20 ]]; then
    echo 'Depart autorise'
else
    echo 'Recharge necessaire'
fi
```

Résultat : `Depart autorise`. Avec `energie=10`, tu prendrais l'autre chemin.

Le bon opérateur pour la bonne lecture Comparer des nombres avec les opérateurs de texte ne déclenche aucune erreur : seul le résultat change.

```
a=10
b=9
[[ "$a" -gt "$b" ]]   # VRAI : lus comme des nombres, 10 est plus grand que 9
[[ "$a" > "$b" ]]     # FAUX : lus comme du texte, « 1 » vient avant « 9 »
```

Retiens la règle : `==` et `!=` pour du texte, `-eq` et `-gt` pour des nombres.

On peut distinguer plusieurs situations avec `elif`. Voici la forme générale, à lire comme un modèle :

```
if premier_test; then
    # commandes si le premier test reussit
elif second_test; then
    # commandes si le premier echoue et que celui-ci reussit
else
    # commandes si aucun test ne reussit
fi
```

Tu peux enchaîner autant de `elif` que nécessaire. Le bloc `else` est facultatif : sans lui, un cas non prévu ne déclenche simplement rien.

Appliqué à l'énergie du vaisseau, en demandant la valeur plutôt qu'en la fixant dans le code :

```
read -p 'Energie disponible : ' energie

if [[ "$energie" -ge 50 ]]; then
    echo 'Grande mission'
elif [[ "$energie" -ge 20 ]]; then
    echo 'Petite mission'
else
    echo 'Recharge'
fi
```

Relance-le trois fois pour parcourir les trois branches :

```
Energie disponible : 60     → Grande mission
Energie disponible : 30     → Petite mission
Energie disponible : 10     → Recharge
```

Bash examine les branches de haut en bas et s'arrête à la première qui réussit : avec `60`, il ne teste même pas `-ge 20`.

Une saisie n'est pas forcément un nombre Réponds `banane` à la question, puis observe :

```
Energie disponible : banane
Recharge
```

Aucune erreur, aucun avertissement. Dans `[[ ... ]]`, un mot qui n'est pas un nombre est lu comme la valeur `0` — donc inférieur à 20. Le script répond quelque chose de faux sans prévenir.

C'est la limite d'un test numérique sur une saisie libre : il faut vérifier la valeur **avant** de la comparer. Le §6 installe ce réflexe — vérifier d'abord, agir ensuite — et le §7 fournit un outil pour le faire, puisqu'un

motif de `case` peut décrire la forme attendue.

4. Tester un fichier

```
mkdir -p ~/atelier-bash/conditions
cd ~/atelier-bash/conditions
echo 'ORION-OK' > billet.txt

if [[ -f "billet.txt" ]]; then
    echo 'Billet trouve'
else
    echo 'Billet absent'
fi
```

Test	Vérification
`-e`	Le chemin existe
`-f`	C'est un fichier ordinaire
`-d`	C'est un dossier
`-r`	Le fichier est lisible pour le processus
`-w`	Le fichier est modifiable pour le processus
`-s`	Le fichier existe et sa taille dépasse zéro

Un dossier n'est pas un fichier ordinaire. Choisis le test adapté plutôt que de tout vérifier avec `-e` : avant de lire un fichier, `-f` puis `-r` ; avant d'y écrire, `-w`.

5. Réagir au résultat d'une commande

Le §1 l'annonçait : `if` ne sait lire qu'un code de retour. On peut donc placer **n'importe quelle commande** après `if`, sans `[[ ... ]]`.

```
if grep -qx 'ORION-OK' billet.txt; then
    echo 'Billet accepte'
else
    echo 'Billet refuse'
fi
```

`grep -q` n'affiche pas les lignes trouvées. `-x` exige une correspondance sur une ligne entière. Le code `0` de `grep` ouvre le premier bloc.

Cet exemple vérifie qu'au moins une ligne est exactement `ORION-OK`. Ce n'est pas une vérification de signature ni un vrai système d'accès.

Même principe pour savoir si un outil est présent sur la machine :

```
if command -v curl > /dev/null; then
    echo 'curl est installe'
else
    echo 'curl est absent'
```

```
fi
```

`command -v` cherche la commande et renvoie `0` si elle existe. `> /dev/null` jette son affichage : seul le code de retour nous intéresse.

6. Vérifier les arguments avant de les utiliser

Contenu de `controle.sh` :

fichier: controle.sh

```
#!/usr/bin/env bash

if [[ "$#" -ne 1 ]]; then
    echo "Usage : $0 fichier" >&2
    exit 2
fi

fichier="$1"
if [[ ! -f "$fichier" ]]; then
    echo "Fichier absent : $fichier" >&2
    exit 2
fi

echo "Fichier disponible : $fichier"
exit 0
```

- `$#` vérifie le nombre d'arguments **avant** la lecture de `$1`.
- `!` inverse un test.
- `>&2` dirige un message vers la sortie d'erreur.
- `exit` termine ce script avec le code choisi.

Vérifie la syntaxe, puis lance les trois cas. Comme au chapitre 03, `$?` se lit **juste après** la commande concernée :

```
bash -n controle.sh

bash controle.sh billet.txt
echo "Code : $?"

bash controle.sh introuvable.txt
echo "Code : $?"

bash controle.sh
echo "Code : $?"
```

Tu obtiens dans l'ordre :

```
Fichier disponible : billet.txt
Code : 0
Fichier absent : introuvable.txt
Code : 2
Usage : controle.sh fichier
Code : 2
```

Le premier cas réussit. Le deuxième est arrêté par la seconde garde, le troisième par la première. Le code `2` est une convention choisie pour ce programme ; d'autres commandes en utilisent d'autres.

7. Choisir parmi plusieurs mots avec case

`case` compare une même valeur à plusieurs motifs successifs. Voici la forme générale, à lire comme un modèle :

```
case "$variable" in
    motif)
        # commandes si la valeur correspond a ce motif
        ;;
    autre_motif)
        # commandes si elle correspond a cet autre motif
        ;;
    *)
        # commandes pour tous les cas restants
        ;;
esac
```

Bash examine les motifs de haut en bas et s'arrête au premier qui correspond.

Appliqué à un choix de poste :

```
choix="radio"
case "$choix" in
    radio)
        echo 'Canal radio ouvert'
        ;;
    reserve)
        echo 'Reserve disponible'
        ;;
    *)
        echo 'Choix inconnu'
        ;;
esac
```

Chaque choix se termine par `)`. `;;` termine sa branche. `*` récupère les autres cas. `esac` ferme la structure.

Une même branche peut accepter plusieurs mots, séparés par `|` :

```
case "$choix" in
    radio | phonie)
        echo 'Canal radio ouvert'
        ;;
    reserve | stock)
        echo 'Reserve disponible'
        ;;
    *)
        echo 'Choix inconnu'
        ;;
esac
```

C'est l'intérêt principal de `case` face à une série de `elif` : regrouper les synonymes sans répéter la condition.

Un motif n'est pas forcément un mot entier : il peut décrire une **forme**. `[0-9]` n'accepte qu'un seul chiffre, `*.txt` qu'un nom finissant par `.txt`. C'est ce qui permet de valider une saisie avant de s'en servir, comme le demandait l'avertissement du §3 :

```
read -p 'Un chiffre de 0 a 9 : ' choix
case "$choix" in
    [0-9])
        echo "Chiffre accepte : $choix"
        ;;
    *)
        echo "Ce n'est pas un chiffre" >&2
        ;;
esac
```

8. Combiner deux vérifications

Dans un test

`&&` signifie « et » : les deux tests doivent réussir.

```
if [[ -f "billet.txt" && -r "billet.txt" ]]; then
    echo 'Le billet est un fichier lisible'
fi
```

`||` signifie « ou » : il suffit que l'un des deux réussisse.

```
destination="Vega"
if [[ "$destination" == "Orion" || "$destination" == "Vega" ]]; then
    echo 'Destination desservie'
else
    echo 'Destination non desservie'
fi
```

Résultat : `Destination desservie`. Avec `destination="Sirius"`, tu obtiendrais l'autre message.

Commence par des tests courts ; si la condition devient difficile à lire, sépare les vérifications et explique chaque erreur séparément.

Entre deux commandes

Les mêmes signes s'emploient aussi **entre deux commandes**, en dehors de tout `[[ ... ]]`. Le sens est identique, appliqué cette fois aux codes de retour du §1 :

- `A && B` : `B` ne s'exécute que si `A` a **réussi**.
- `A || B` : `B` ne s'exécute que si `A` a **échoué**.

```
mkdir -p rapports && echo 'Dossier pret'
grep -qx 'AUTRE-STATION' billet.txt || echo 'Billet non conforme' >&2
```

La première ligne n'affiche son message que si le dossier a bien été créé. La seconde signale un problème uniquement quand `grep` ne trouve rien.

C'est une écriture courte pour un `if` à une seule branche :

```
# ces deux ecritures font la meme chose
```

```
if ! grep -qx 'AUTRE-STATION' billet.txt; then
    echo 'Billet non conforme' >&2
fi

grep -qx 'AUTRE-STATION' billet.txt || echo 'Billet non conforme' >&2
```

Réserve-la aux cas courts : dès qu'il y a plusieurs commandes à enchaîner, le `if` reste plus lisible.

À retenir

- `if` lit le **code de retour** d'une commande : `0` ouvre le bloc `then`. C'est l'inverse de la convention d'autres langages.
- Une valeur est toujours du texte : `==` et `!=` la comparent comme du texte, `-eq` et `-gt` comme un nombre.
- Texte : `-z` pour une valeur vide, `-n` pour une valeur remplie.
- Fichiers : `-f` et `-d` pour le type, `-r` et `-w` pour les droits, `-s` pour un contenu non vide.
- N'importe quelle commande peut servir de test : `grep -q`, `command -v`, ou un script à toi.
- Vérifie les arguments et les fichiers avant leur utilisation ; prévois un message sur `>&2` et un code de retour avec `exit`.
- `case` regroupe plusieurs mots par branche avec `|`.
- `&&` exige les deux, `||` se contente d'un seul — dans un test comme entre deux commandes, où `A || B` ne lance `B` que si `A` a échoué.

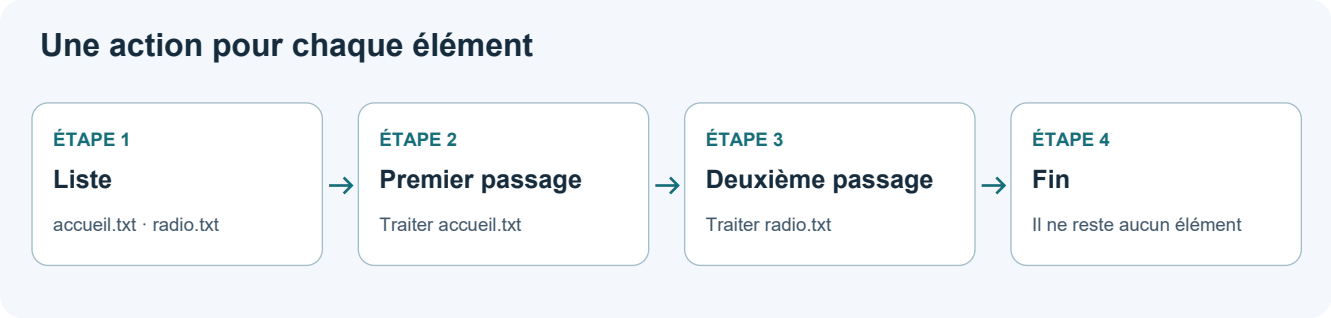
À toi : quel test différencie un dossier d'un fichier ? Quelle branche exécute `case` pour un mot inconnu ? Pourquoi `if grep -q ...` fonctionne-t-il sans `[[ ... ]]` ?

Passer au TP

Chapitre 05 — Les boucles

Objectif Répéter une action sur une liste ou un groupe de fichiers, sans recopier les mêmes commandes.

1. Une action, plusieurs éléments



La même action pour chaque fichier

Une boucle `for` prend les éléments **un par un**. À chaque passage, une variable contient l'élément courant. Voici la forme générale, à lire comme un modèle :

```
for variable in element1 element2 element3; do
    # commandes utilisant "$variable"
done
```

Appliquée à une liste de prénoms :

```
for personne in Mila Noe Vega; do
    echo "Bonjour $personne"
done
```

Résultat : trois lignes, une par personne. `do` ouvre le corps de la boucle ; `done` le ferme.

Un élément peut contenir des espaces :

```
for poste in "salle radio" "poste de pilotage"; do
    echo "Controle : $poste"
done
```

Les guillemets conservent ici **deux éléments**, pas six mots.

2. Répéter un nombre de fois

Une liste, ou une plage

Les éléments peuvent être des nombres :

```
for numero in 1 2 3; do
    echo "Test numero $numero"
done
```

Au-delà de quelques valeurs, écris une **plage** plutôt que la liste complète :

```
for numero in {1..10}; do
    echo "Test numero $numero"
done
```

`{1..10}` est remplacé par Bash avant l'exécution : la boucle reçoit bien dix éléments. Tu as déjà croisé cette écriture dans les démos du chapitre 01 — c'est une facilité de Bash que `sh` ne connaît pas.

Une plage n'accepte pas de variable Bash remplace `{1..10}` **avant** de lire les variables. Avec une borne variable, le remplacement n'a donc pas lieu :

```
max=5
for numero in {1..$max}; do echo "$numero"; done
```

```
{1..5}
```

Un seul passage, avec le texte littéral. Pour une borne variable, utilise une boucle `while` comme au §4.

Compter les passages

Une boucle peut aussi tenir le compte de ce qu'elle a traité :

```
compteur=0
for objet in radio casque batterie; do
    compteur=$((compteur + 1))
    echo "$compteur : $objet"
done
echo "Objets controles : $compteur"
```

On initialise le compteur **avant** la boucle, on augmente sa valeur **dans** la boucle, on le lit **après**. Le résultat final est 3.

3. Parcourir des fichiers avec un motif

C'est l'usage le plus fréquent de `for` : agir sur tous les fichiers qui portent un certain nom.

Prépare quelques données :

```
mkdir -p ~/atelier-bash/boucles/messages
cd ~/atelier-bash/boucles
echo 'Bienvenue' > messages/accueil.txt
echo 'Radio active' > "messages/radio du jour.txt"
echo 'Image fictive' > messages/image.png
```

```
for fichier in messages/*.txt; do
    if [[ -f "$fichier" ]]; then
        echo "Lecture de $fichier"
        cat "$fichier"
    fi
done
```

`*.txt` est un **motif de noms** : Bash le remplace par la liste des noms qui correspondent, avant que la boucle démarre. Le fichier `image.png` n'est pas sélectionné. C'est ce mécanisme qui rend `for` utile au quotidien — tu n'as pas à connaître les noms à l'avance.

Le test `-f` évite de traiter un dossier. Il gère aussi le cas où **aucun** nom ne correspond : avec les réglages habituels de Bash, le motif reste alors un texte littéral, qui n'est pas un fichier.

```
for fichier in messages/*.csv; do echo "[$fichier]"; done
```

```
[messages/*.csv]
```

Un seul passage, avec le motif non remplacé : sans le test `-f`, la boucle essaierait de lire un fichier nommé `*.csv`.

La bonne place des guillemets Écris `"$dossier"/*.txt` si le dossier vient d'une variable. La variable est protégée ; le `*` reste disponible pour la recherche. Ne fais pas une boucle sur le résultat de `ls` : les noms contenant des espaces seraient difficiles à traiter correctement.

4. Une boucle while répète tant que le test réussit

Là où `for` parcourt une liste connue d'avance, `while` répète tant qu'une condition reste vraie. Forme générale :

```
while test; do
    # commandes repetees tant que le test reussit
    # l'une d'elles doit faire evoluer le test
done
```

```
compteur=3
while [[ "$compteur" -gt 0 ]]; do
    echo "Depart dans $compteur"
    compteur=$((compteur - 1))
done
echo 'Decollage'
```

La valeur descend de 3 à 1. Quand elle vaut 0, le test échoue et la boucle se termine.

La ligne `compteur=$((compteur - 1))` est celle qui fait évoluer le test. Si tu l'oublies, la condition ne changera jamais : **Ctrl+C** permet d'interrompre une boucle sans fin.

5. Lire un fichier ligne par ligne

```
printf 'Mila Vega\nNoe\n' > equipe.txt
while IFS= read -r personne; do
    echo "Equipier : $personne"
done < equipe.txt
```

Nous utilisons ici `printf` plutôt qu'`echo` pour écrire **deux lignes en une commande** : c'est le cas de figure du chapitre 02 où `printf` garde l'avantage.

- `read` lit une ligne à chaque passage, et la boucle s'arrête quand il n'y en a plus.
- `< equipe.txt` fournit le fichier comme entrée à la boucle.
- `IFS=` et `-r` protègent le contenu de la ligne.

Pourquoi `IFS=` et `-r` ? Sur des lignes ordinaires, ils ne changent rien — c'est pour cela qu'on les oublie. Leur effet n'apparaît que sur un contenu moins sage :

```
printf '%s\n' '   Mila Vega   ' 'C:\Orion\radio' > brut.txt
```

Lecture	Première ligne	Seconde ligne
`while IFS= read -r ligne`	`[ Mila Vega ]`	`[C:\Orion\radio]`
`while read ligne`	`[Mila Vega]`	`[C:Orionradio]`

Sans `IFS=`, les espaces de début et de fin sont retirés. Sans `-r`, les antislashs sont pris pour des caractères d'échappement et disparaissent. Les données réelles — noms de fichiers, chemins, champs recopiés — contiennent ce genre de surprises.

Retiens `while IFS= read -r ligne; do ... done < fichier` comme un bloc : c'est la façon correcte de lire un fichier ligne par ligne.

Ici, les lignes se terminent toutes par un retour à la ligne, créé avec `\n`. Pour traiter aussi une dernière ligne sans ce retour, on peut compléter la condition ; ce cas est proposé en bonus du TP.

6. Passer un élément ou arrêter la boucle

`continue` passe directement à l'élément suivant. `break` quitte la boucle.

```
for message in INFO IGNORER ALERTE FIN; do
    if [[ "$message" == "IGNORER" ]]; then
        continue
    fi
    if [[ "$message" == "FIN" ]]; then
        break
    fi
    echo "Message : $message"
done
```

Résultat : `INFO` puis `ALERTE`. `IGNORER` est sauté, et `FIN` interrompt le parcours avant `echo`. Commence par une boucle simple ; ajoute ces instructions seulement si elles rendent le parcours plus clair.

7. Exemple complet : copier les messages

Dans le dossier `boucles`, crée `copier.sh` :

fichier: copier.sh

```
#!/usr/bin/env bash

if [[ "$#" -gt 1 ]]; then
    echo "Usage : $0 [dossier]" >&2
    exit 2
fi

source="${1:-messages}"
if [[ ! -d "$source" ]]; then
    echo "Dossier absent : $source" >&2
    exit 2
fi
```

```
destination="$source/copies"
if ! mkdir -p -- "$destination"; then
    echo 'Impossible de creer la destination' >&2
    exit 1
fi

compteur=0
for fichier in "$source"/*.txt; do
    if [[ ! -f "$fichier" ]]; then
        continue
    fi
    if ! cp -- "$fichier" "$destination/"; then
        echo "Copie impossible : $fichier" >&2
        exit 1
    fi
    compteur=$((compteur + 1))
done

echo "Copies realisees : $compteur"
```

Ce script réunit tout le chapitre : une garde sur les arguments, un motif, un `continue`, un compteur, et un code de sortie par type d'erreur.

`--` marque la fin des options pour `mkdir` et `cp`. Un nom commençant par `-` ne sera donc pas pris pour une option. `cp` copie les fichiers sans supprimer les originaux.

```
bash -n copier.sh
bash copier.sh messages
ls messages/copies
```

Deux fichiers doivent être copiés. Ce petit outil remplace les copies de même nom lors d'un nouveau lancement : ce n'est pas un système de sauvegarde avec historique.

8. Écrire une fonction

Une fonction donne un nom à un groupe de commandes. Tu peux l'appeler plusieurs fois, ce qui évite de répéter les mêmes lignes dans une boucle ou dans un script :

```
saluer() {
    local personne="${1:-equipier}"
    echo "Bienvenue $personne"
}

saluer Mila
saluer Vega
saluer
```

Résultat : `Bienvenue Mila`, `Bienvenue Vega`, puis `Bienvenue equipier`.

- Les arguments de la fonction se lisent avec `$1`, `$2`, comme ceux d'un script.
- `local` réserve la variable à cette fonction : elle n'écrase pas une variable du même nom ailleurs dans le script.
- La fonction doit être **définie avant** d'être appelée : place-la en haut du fichier.

Les TP de synthèse et le TP final utilisent une fonction pour écrire les titres d'un rapport : ce n'est pas une notion facultative.

À retenir

- `for variable in liste; do ... done` parcourt des éléments connus ; `while test; do ... done` répète tant qu'une condition réussit.
- La plage `{1..10}` évite d'écrire une longue liste de nombres, mais n'accepte pas de borne variable.
- Un compteur s'initialise **avant** la boucle et s'incrémente **dans** la boucle avec `$((compteur + 1))`.
- Le motif `*.txt` est remplacé par les noms correspondants avant le démarrage de la boucle. S'il ne correspond à rien, il reste littéral : garde un test `-f`.
- `while IFS= read -r ligne; do ... done < fichier` est la façon correcte de lire un fichier ligne par ligne.
- `continue` saute un élément, `break` arrête la boucle.
- Une fonction nomme un groupe de commandes ; `local` y réserve une variable.
- Protège les noms avec `"$variable"`, et pense à tester une liste vide, un nom composé et un dossier absent.

À toi : que se passe-t-il si le motif `*.txt` ne correspond à aucun fichier ? Pourquoi `IFS=` et `-r` ne changent-ils rien sur `Mila Vega`, mais tout sur `C:\Orion\radio` ?

Passer au TP

Chapitre 06 — Traiter du texte avec sed et awk

Objectif Transformer un fichier texte, sélectionner des colonnes et calculer un total, sans modifier l'original par erreur.

1. Choisir l'outil selon la tâche

Besoin	Outil
Garder des lignes contenant un mot	`grep`
Remplacer un mot ou sélectionner des lignes	`sed`
Lire des champs et calculer	`awk`

Les trois outils lisent le texte. **Par défaut, ils ne changent pas le fichier lu.** Ils envoient un résultat sur leur sortie standard.

2. Préparer un petit inventaire

Dans Bash sur Debian, colle le bloc complet jusqu'à `EOF` :

```
mkdir -p ~/atelier-bash/texte
cd ~/atelier-bash/texte
cat > stock.csv <<'EOF'
produit;categorie;quantite;prix
clavier;informatique;8;25
souris;informatique;12;15
cahier;papeterie;20;3
ecran;informatique;4;180
stylo;papeterie;30;2
EOF
```

`<<'EOF'` fournit les lignes suivantes à `cat`, sans interpréter leur contenu. `>` écrit le résultat dans `stock.csv`. La dernière ligne `EOF` doit être seule.

Cette écriture remplace avantageusement les `echo` du chapitre 05 dès qu'il y a plus de deux ou trois lignes à écrire : le contenu du fichier apparaît tel quel dans le script, sans guillemets ni `\n` à gérer.

Ce fichier utilise `;` pour séparer quatre champs. La première ligne est un en-tête. C'est un format simple : aucun champ ne contient de point-virgule ni de saut de ligne. Le `awk` de ce cours ne remplace pas un lecteur CSV capable de traiter tous les cas de champs entre guillemets.



Une ligne découpée en quatre champs

3. Remplacer du texte avec sed

`sed` signifie *stream editor*, éditeur de flux. Il lit un fichier **ligne par ligne**, applique à chaque ligne les commandes que tu lui donnes, et écrit le résultat sur sa sortie standard. Il ne charge pas le fichier en mémoire et ne l'ouvre pas dans une fenêtre : il n'y a rien à faire défiler, rien à enregistrer.

C'est ce qui le rend utile là où un éditeur ne suffit pas : remplacer un mot dans cinquante fichiers de configuration, retirer les lignes de commentaire d'un fichier avant de le traiter, ou extraire les lignes 200 à 250 d'un journal de plusieurs gigaoctets. Les tâches typiques en administration sont le remplacement en masse, la suppression de lignes et l'extraction d'une portion de fichier.

Deux points à retenir dès maintenant : `sed` travaille **sur une ligne à la fois** — il ne sait pas ce qu'il y a trois lignes plus loin — et il **ne modifie pas le fichier lu**, sauf si tu l'y forces avec l'option `-i` (§9).

```
sed 's/informatique/materiel/g' stock.csv
```

Lis l'expression de gauche à droite :

Partie	Rôle
`s`	Substitution
`informatique`	Texte recherché dans cet exemple
`materiel`	Texte de remplacement
`g`	Toutes les occurrences sur chaque ligne

Les guillemets simples protègent l'expression du shell. La recherche de `sed` utilise des expressions régulières ; ici, nous cherchons un mot sans caractère spécial.

À quoi sert vraiment le `g` ? Sur l'exemple ci-dessus, il ne sert à rien : aucune ligne ne contient deux fois `informatique`, donc avec ou sans `g` le résultat est identique. Son effet n'apparaît que lorsqu'un motif revient plusieurs fois sur la même ligne — le point-virgule, par exemple :

```
echo 'stylo;papeterie;30;2' | sed 's/;/ | /'
echo 'stylo;papeterie;30;2' | sed 's/;/ | /g'
```

```
stylo | papeterie;30;2
stylo | papeterie | 30 | 2
```

Sans `g`, `sed` ne remplace que **la première** occurrence de chaque ligne. C'est l'oubli le plus fréquent avec `s///`.

Mémo — les variantes de `s///`

Écriture	Effet	Exemple	Résultat
`s/a/X/`	la **première** occurrence de chaque ligne	`echo 'a;b;c' | sed 's/;/-/'`	`a-b;c`
`s/a/X/g`	**toutes** les occurrences	`echo 'a;b;c' | sed 's/;/-/g'`	`a-b-c`
`s/a/X/2`	la **2e** occurrence seulement	`echo 'a;b;c;d' | sed 's/;/-/2'`	`a;b-c;d`
`s/a/X/i`	sans tenir compte de la casse	`echo 'Radio RADIO' | sed 's/radio/X/gi'`	`X X`
`s/a/[&]/`	& rappelle le **texte trouvé**	`echo 'clavier' | sed 's/clavier/[&]/'`	`[clavier]`
`s/(…)/\1/`	`\1` rappelle un **groupe capturé** (avec `-E`)	`echo 'clavier;8' | sed -E 's/^(.+);(.+)$/\2 x \1/'`	`8 x clavier`

Le délimiteur n'est pas forcément `/` **Le caractère qui suit le `s` devient le délimiteur.** Dès que le motif contient des chemins, changer de délimiteur évite d'échapper chaque slash :

```
sed 's/\/etc\/apt\//\/srv\/miroir\//' chemins.txt   # illisible
sed 's|/etc/apt/|/srv/miroir/|' chemins.txt         # identique, lisible
```

`s#…#…#` et `s,…,…,` fonctionnent aussi. Choisis un caractère absent du motif et du remplacement.

Pour conserver le résultat :

```
sed 's/informatique/materiel/g' stock.csv > stock-corrige.csv
cat stock-corrige.csv
cat stock.csv
```

Le nouveau fichier contient `materiel`. L'original contient toujours `informatique`.

Deux noms de fichiers différents N'écris jamais `sed ... stock.csv > stock.csv`. Le shell viderait le fichier de destination avant que `sed` puisse le lire.

4. Choisir ou retirer des lignes

```
sed -n '2,4p' stock.csv
```

`-n` coupe l'affichage automatique ; `2,4p` affiche les lignes 2 à 4 incluses. Tu obtiens clavier, souris et cahier.

```
sed '1d' stock.csv
```

`1d` retire la première ligne de la **sortie**. Le fichier n'est pas modifié.

```
sed '/papeterie/d' stock.csv
```

Ce dernier exemple retire de la sortie les lignes contenant `papeterie`.

Mémo — une commande `sed` se lit `adresse` + `action`

Dans `sed '2,4p'`, `2,4` est l'**adresse** (quelles lignes) et `p` est l'**action** (quoi en faire). Les deux parties se combinent librement : `1d`, `/papeterie/d`, `$p`, `2,4s/a/X/`...

Les adresses les plus utiles — exemples sur notre `stock.csv` de 6 lignes :

Adresse	Désigne	Exemple
`3`	la ligne 3	`sed -n '3p'`
`2,4`	les lignes 2 à 4 incluses	`sed -n '2,4p'`
`2,$`	de la ligne 2 à la fin	`sed -n '2,$p'`
`$`	la dernière ligne	`sed -n '$p'`
`/motif/`	les lignes contenant le motif	`sed -n '/papeterie/p'`
`/a/,/b/`	de la 1re ligne a à la 1re ligne b	`sed -n '/souris/,/ecran/p'`
`2,+2`	la ligne 2 et les 2 suivantes	`sed -n '2,+2p'`
`0~2`	une ligne sur deux	`sed -n '0~2p'`
`1!`	**toutes sauf** la ligne 1 (`!` = négation)	`sed -n '1!p'`
(aucune)	toutes les lignes	`sed 's/a/X/'`

Les actions les plus utiles :

Action	Rôle	Exemple	Effet
`s///`	substituer	`sed 's/a/X/'`	remplace
`p`	afficher (avec `-n`)	`sed -n '2p'`	ligne 2
`d`	supprimer de la sortie	`sed '1d'`	sans l'en-tête
`q`	quitter à cette ligne	`sed '3q'`	les 3 premières lignes
`=`	afficher le **numéro** de ligne	`sed -n '/ecran/='`	5
`a texte`	ajouter une ligne **après**	`sed '1a ---'`	insère après l'en-tête
`i texte`	insérer une ligne **avant**	`sed '1i # titre'`	ajoute un titre

Action	Rôle	Exemple	Effet
`c texte`	remplacer la ligne entière	`sed '2c nouveau'`	écrase la ligne 2
`y/abc/ABC/`	translittérer caractère à caractère	`echo abc | sed 'y/abc/ABC/'`	ABC

Mémo — les options de `sed`

Option	Rôle
`-n`	n'affiche rien automatiquement — va de pair avec p
`-e`	enchaîner plusieurs expressions : `sed -n -e '1p' -e '3p'`
`-E`	expressions régulières **étendues** : `(…)`, `{2}`, `+`, `?` sans antislash
`-i`	modifie le fichier **sur place** — voir le §9, à utiliser avec `-i.bak`
`-f`	lire les commandes depuis un fichier de script `sed`

`q` s'arrête vraiment `sed '3q'` ne se contente pas de filtrer : `sed` **quitte** à la ligne 3 et ne lit pas la suite. C'est ce qui rend `sed '1000q'` beaucoup plus rapide que `head -1000` sur un fichier de plusieurs gigaoctets.

5. Lire les champs avec awk

`awk` porte les initiales de ses trois auteurs (Aho, Weinberger, Kernighan). Comme `sed`, il lit le fichier ligne par ligne — mais au lieu de voir chaque ligne comme du texte, il la **découpe en champs** numérotés `$1`, `$2`, `$3`… Un fichier devient ainsi un tableau de lignes et de colonnes.

C'est ce découpage qui fait tout l'intérêt de l'outil : il permet de désigner une colonne précise, de filtrer sur sa valeur et de calculer. `awk` sait additionner, multiplier, compter et stocker des résultats intermédiaires, ce que `sed` ne sait pas faire. Les tâches typiques sont l'extraction de colonnes, le filtrage sur un critère chiffré, le calcul de totaux et le regroupement par catégorie.

En pratique, tu le croiseras sur tout ce qui arrive en colonnes : un export CSV, la sortie de `ps` ou de `df`, un journal d'accès Apache, un fichier `/etc/passwd`.

La règle pour choisir : **si tu raisonnes en lignes et en mots, c'est `sed` ; si tu raisonnes en colonnes et en chiffres, c'est `awk`.**

Un programme `awk` s'écrit toujours sur le même modèle :

```
awk -F 'separateur' 'condition { action }' fichier
```

La **condition** choisit les lignes, l'**action** dit quoi en faire. Appliqué à notre inventaire :

```
awk -F ';' 'NR > 1 { print $1, $3 }' stock.csv
```

- `-F ';'` fixe le séparateur de champs.
- `NR` est le numéro de la ligne en cours.
- `NR > 1` saute l'en-tête : c'est la condition.
- `{ print $1, $3 }` est l'action : `$1` est le produit, `$3` sa quantité.

Résultat :

```
clavier 8
souris 12
cahier 20
ecran 4
stylo 30
```

Les `$1` et `$3` appartiennent ici à **awk**, pas à Bash : les guillemets simples empêchent Bash de les remplacer.

Les deux morceaux sont facultatifs C'est ce qui rend les programmes `awk` si variés d'aspect :

Programme	Sens
`'NR > 1 { print $1 }'`	Condition **et** action
`'{ print $1 }'`	Pas de condition → l'action s'applique à **toutes** les lignes
`'NR > 1'`	Pas d'action → awk affiche la ligne entière

Garde ce tableau en tête : tu croiseras les trois formes, y compris au §8 de ce chapitre.

Mémo — les variables d'`awk`

Elles s'écrivent **sans** `$` : les `$` désignent des colonnes, ces variables n'en sont pas.

Variable	Contient	Exemple sur `stock.csv`
`NR`	numéro de la ligne en cours	`awk 'END { print NR }'` → `6`
`NF`	nombre de champs de la ligne	`awk -F ';' 'NR==2 { print NF }'` → `4`
`$0`	la ligne entière	identique à `cat`
`$1`, `$2`…	les champs, de gauche à droite	`$1` = produit
`$NF`	le **dernier** champ, quel que soit leur nombre	`$NF` = prix
`$(NF-1)`	l'avant-dernier champ	`$(NF-1)` = quantité
`FS`	séparateur d'**entrée** — équivaut à `-F`	`awk 'BEGIN { FS=";" } …'`
`OFS`	séparateur de **sortie** (espace par défaut)	`awk -F ';' -v OFS=' | ' …`

Variable	Contient	Exemple sur `stock.csv`
`FILENAME`	nom du fichier lu	utile avec plusieurs fichiers

Passer une valeur du shell à `awk` Les guillemets simples empêchent Bash de substituer ses variables. L'option `-v` est la façon propre de faire entrer une valeur :

```
seuil=10
awk -F ';' -v seuil="$seuil" 'NR>1 && $3 < seuil { print $1 }' stock.csv
```

Dans le programme `awk`, `seuil` s'écrit **sans** `$` : c'est une variable `awk`, pas une variable Bash.

Mémo — les opérateurs de condition

Opérateur	Sens	Exemple
`== !=`	égal / différent	`$2 == "papeterie"`
`< <= > >=`	comparaisons	`$3 < 10`
`~`	correspond à une regex	`$1 ~ /^c/`
`!~`	ne correspond pas	`$2 !~ /papeterie/`
`&& || !`	et / ou / non	`NR>1 && $3 < 10`

Mémo — les fonctions les plus utiles

Fonction	Rôle	Exemple	Résultat
`length(s)`	longueur	`length("clavier")`	`7`
`toupper(s)` / `tolower(s)`	casse	`toupper("clavier")`	`CLAVIER`
`substr(s, debut, n)`	extraire	`substr("clavier", 1, 4)`	`clav`
`int(n)`	partie entière	`int(7.9)`	`7`
`gsub(/motif/, "X")`	remplacer dans `$0`	`gsub(/;/, "-")`	`a-b-c`
`split(s, t, "-")`	découper dans un tableau	`split("a-b-c", t, "-")`	`3`, et `t[2]` = `b`

Lequel des deux `awk` ? Sur Debian, `awk` est **mawk** par défaut, pas **gawk**. Les tableaux ci-dessus fonctionnent sur les deux. En revanche, `gensub()`, `asort()`, `IGNORECASE` ou `PROCINFO["sorted_in"]` sont propres à gawk : beaucoup d'exemples trouvés en ligne les utilisent sans le préciser. Vérifie avec `awk --version`.

6. Filtrer avec une condition

```
awk -F ';' 'NR > 1 && $3 < 10 { print $1, $3 }' stock.csv
```

La ligne doit être après l'en-tête **et** sa quantité doit être inférieure à 10. Résultat : `clavier 8`, puis `ecran 4`.

> Un champ n'est pas forcément un nombre `awk` compare numériquement quand les deux côtés ressemblent à des nombres. Sinon, il compare comme du texte — sans prévenir :

```
champ [8]     → selectionne     (8 < 10)
champ [80]    → ecarte          (80 > 10)
champ [huit]  → ecarte          compare comme du texte
champ []      → SELECTIONNE     un champ vide compte pour 0
```

> Le dernier cas est le piège : une quantité manquante est signalée comme un stock faible. Avant de filtrer un fichier qui n'est pas le tien, regarde-le avec `sed -n '1,5p'`.

7. Calculer un total

```
awk -F ';' 'NR > 1 { total += $3 } END { print total }' stock.csv
```

À chaque produit, `total += $3` ajoute sa quantité. Après la dernière ligne, `END` affiche la somme. Résultat : **74 articles**.

Tu remarques qu'on n'initialise pas `total`, contrairement au compteur du chapitre 05. C'est une particularité d'`awk` : une variable jamais définie vaut la chaîne vide, et `0` dès qu'on s'en sert comme nombre. En Bash, il aurait fallu écrire `total=0` avant la boucle.

Pour calculer la valeur de l'inventaire :

```
awk -F ';' 'NR > 1 { total += $3 * $4 } END { print total }' stock.csv
```

Chaque quantité est multipliée par son prix unitaire. Résultat : **1220**.

Le `END` d'`awk` n'est pas le `done` d'une boucle Bash : nous utilisons deux langages différents, chacun avec sa syntaxe.

8. Relier deux outils

```
sed '1d' stock.csv | awk -F ';' '{ total += $3 } END { print total }'
```

`sed` retire l'en-tête, puis le tube transmet les produits à `awk`. On retrouve 74.

Remarque le programme `awk` : il n'a plus de condition devant `{ total += $3 }`, puisque l'en-tête a déjà été écarté par `sed`. C'est la deuxième forme du tableau du §5 — l'action s'applique à toutes les lignes reçues.

C'est une autre façon de faire ; le `awk` du §7 suffit quand il sait filtrer lui-même l'en-tête.

9. Modifier un fichier avec une sauvegarde

Jusqu'ici, aucune commande n'a touché à `stock.csv`. Pour modifier un fichier sur place, `sed` a l'option `-i` — à n'utiliser qu'avec une sauvegarde, et d'abord sur une copie :

```
cp stock.csv stock-test.csv
sed -i.bak 's/informatique/materiel/g' stock-test.csv
cat stock-test.csv
cat stock-test.csv.bak
```

Sur Debian, `-i.bak` modifie `stock-test.csv` et conserve l'ancienne version dans `stock-test.csv.bak`. Ce comportement est celui de GNU sed ; ne suppose pas que les options sont identiques sur tous les systèmes.

Le défi supplémentaire du TP demande cette option : ce n'est pas une notion facultative.

À retenir

- `sed` transforme ou sélectionne du texte ; `awk` travaille avec des champs.
- Une commande `sed` se lit **adresse + action** : `2,4p`, `1d`, `/papeterie/d`, `$p`. Les mémos des §3, §4 et §5 regroupent les adresses, les actions, les options et les variables à connaître.
- `cat > fichier <<'EOF'` écrit plusieurs lignes d'un coup, telles quelles.
- `s/recherche/remplacement/g` : sans le `g`, seule la **première** occurrence de chaque ligne est remplacée.
- `sed -n '2,4p'` garde des lignes, `sed '1d'` et `sed '/motif/d'` en retirent — toujours sur la sortie, jamais sur le fichier.
- Un programme `awk` s'écrit `'condition { action }'`. Sans condition, l'action vaut pour toutes les lignes ; sans action, la ligne entière est affichée.
- `-F ';'` fixe le séparateur, `NR` donne le numéro de ligne, `NR > 1` saute l'en-tête, `END` s'exécute après la dernière ligne.
- Une variable `awk` n'a pas besoin d'être initialisée : elle vaut `0` dès le premier calcul.
- Identifie le séparateur, l'en-tête et les champs vides avant de calculer.
- Regarde d'abord le résultat, puis écris dans un nouveau fichier. `sed -i.bak` seulement en connaissance de cause.

À toi : que désigne `$4` dans notre inventaire ? Pourquoi utilisons-nous `NR > 1` ? Pourquoi des guillemets simples autour du programme `awk` ? Que ferait `sed 's/;/,/'` sans le `g` sur une ligne de ce fichier ?

Passer au TP 06 bis (pas à pas) · TP 06 avancé